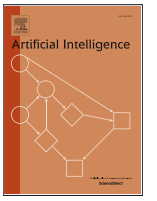
Artificial Intelligence 362 (2027) 104655



Contents lists available at ScienceDirect

Artificial Intelligence

journal homepage: www.elsevier.com/locate/artint



Constraint-search collaborative genetic algorithm for Bayesian network structure learning

Zuowei He [a,b], Hengyang Lu [a], Shuwei Zhu [a], Yunyang Shi [a], Wei Fang [a,*], Kalyanmoy Deb [c]

[a] School of Artificial Intelligence and Computer Science, Jiangnan University, No 1800, Wuxi, 214122, Jiangsu, China
[b] Laboratory for Advanced Computing and Intelligence Engineering, Wuxi, China
[c] Department of Computer Science, University of Alabama, Tuscaloosa, 35487, Alabama, USA

ARTICLE INFO

Keywords:
Bayesian network
Structure learning
Causal discovery
Genetic algorithm
Adaptive constraint
Score memoization

ABSTRACT

Bayesian networks (BNs) serve as a powerful tool for causal learning, yet inferring optimal causal structures from observational data remains a challenging problem. Hybrid structure-learning methods provide a practical paradigm for balancing search efficiency and structural accuracy. However, existing hybrid methods typically treat constraint and structural search as decoupled stages, resulting in the permanent loss of potential causal relationships, while redundant fitness evaluations further limit exploration efficiency. To address these issues, this paper proposes a Constraint-Search Collaborative Genetic Algorithm (CS-CGA). The collaboration is realized through two core improvements: a progressive constraint relaxation mechanism that restores the possibility of discovering mistakenly pruned candidate causal edges, and an exploration-enhanced adaptive mutation operator that enhances the exploration capability for these potential relationships. Furthermore, a hierarchical score memoization strategy is designed to accelerate the evaluation of BN structures. Experiments on eight benchmark BNs demonstrate that CS-CGA achieves favorable structural accuracy and convergence speed compared with representative evolutionary, SS, and hybrid BNSL methods, while also exhibiting robust transferability.

1. Introduction

The discovery of causal relationships is crucial for advancing from artificial narrow intelligence to artificial general intelligence [1]. Bayesian networks (BNs) are commonly employed to encode causal relationships among variables [2], which are represented by directed acyclic graphs (DAGs) and conditional probability tables (CPTs). Owing to their transparency and interpretability, BNs have been applied in fields such as brain connectivity analysis [3], human gesture tracking [4], traffic risk assessment [5], and epidemiology [6]. Bayesian network structure learning (BNSL) is essential for constructing complete BN models, aiming to infer an optimal DAG structure that represents conditional dependencies among variables and may support causal interpretation under appropriate assumptions [7]. However, as the number of variables increases, the number of possible DAGs grows super-exponentially, which makes the identification of an optimal structure an NP-hard problem [8]. Existing methods can be broadly classified into three categories: constraint-based (CB), score-and-search (SS), and hybrid methods based on CB and SS.

* Corresponding author.
E-mail addresses: 7233115008@stu.jiangnan.edu.cn (Z. He), luhengyang@jiangnan.edu.cn (H. Lu), zhushuwei@jiangnan.edu.cn (S. Zhu), yunyang.shi@jiangnan.edu.cn (Y. Shi), fangwei@jiangnan.edu.cn (W. Fang), kdeb@ua.edu (K. Deb).

https://doi.org/10.1016/j.artint.2026.104655
Received 12 March 2026; Received in revised form 28 August 2026; Accepted 30 October 2026
Available online 2 November 2026
0004-3702/© 2026 Published by Elsevier B.V.

CB methods construct BN equivalence classes by identifying conditional independence (CI) relations in the data. Representative algorithms include PC [9], Incremental Association Markov Blanket (IAMB) [10], and Recursive Autonomy Identification (RAI) [11]. Although CB methods possess solid statistical foundations, their performance relies on the faithfulness assumption and is prone to errors when unobserved variables or noisy data are present [12,13]. In contrast, SS methods [14,15] evaluate candidate DAGs using scoring functions that quantify the fit between the structure and the data, then search for the highest-scoring DAG within the space of all possible BNs. The SS methods typically employ traditional heuristic algorithms [16,17] or evolutionary algorithms (EAs) [18,19]. However, the former are prone to premature convergence, while the latter often incur prohibitive computational costs in the vast DAG search space.

Hybrid methods combine the strengths of CB and SS methods [20–22]. CI tests of hybrid approaches are typically used to construct a superstructure, which constrains the subsequent SS phase to a reduced search space. Classic hybrid algorithms include Max-Min Hill Climbing (MMHC) [23] and SaiyanH [24]. To further enhance global exploration capabilities, recent studies have employed EAs, particularly Genetic Algorithms (GAs), in the hybrid framework. GAs contribute to the efficiency and quality of structure learning by enabling broad exploration while supporting exploitation through selection and crossover. Notable methods include hybrid-SLA [25], MIGA [26], and MAGABN [19], which leverage population-based search to mitigate local optima and improve the quality of structure learning.

Despite these advances, the CB phase in hybrid methods is often treated as a static preprocessing step that defines the search space. Candidate causal edges wrongly pruned by fixed significance thresholds cannot be reconsidered during the SS phase [27]. The decoupling of the constraint and search fundamentally limits the discovery of optimal structures. However, expanding the search space without a corresponding search strategy is blind, which commonly leads to low efficiency and stagnation in local optima. Moreover, although decomposable functions theoretically allow score memoization to accelerate fitness assessment [28], as the search space expands, the exponential growth in parent-set combinations slows retrieval and consumes excessive memory.

To overcome these limitations, this paper proposes a Constraint-Search Collaborative Genetic Algorithm (CS-CGA) accelerated by hierarchical score memoization. The main contributions are as follows:

1. Unlike traditional hybrid methods that decouple the CB and SS phases, CS-CGA establishes a collaborative process in which the constraint boundaries and the evolutionary search mutually adapt to optimize structure learning.
2. A progressive constraint relaxation (PCR) mechanism is proposed to dynamize the search space. The initial search space is a high-confidence superstructure to ensure a robust initialization, and is then progressively relaxed based on evolutionary state to recover potential causal edges that are mistakenly pruned.
3. An exploration-enhanced adaptive mutation (EEAM) operator is designed to adjust mutation strategies according to the evolutionary state. Each individual is mutated within the high-confidence superstructure to ensure local convergence, while duplicate individuals undergo a second mutation in the relaxed superstructure to capture potential causal edges further.
4. A hierarchical score memoization (HSM) strategy for decomposable scores is constructed to accelerate fitness evaluation, which alleviates the computational burden of evaluating candidate DAGs through the structural memoization of global topology hashing and local parent-set caching.

The remainder of the paper is organized as follows. Section 2 explains BNSL preliminaries, and Section 3 reviews related work. Section 4 presents the proposed method in detail. Section 5 reports experimental results and comparative analyses. Section 6 concludes the paper and discusses future research directions.

2. Preliminaries

2.1. Problem statement

The structure of a BN is represented as a DAG, denoted as $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where the set of nodes $\mathcal{V}$ represents random variables, and the set of directed edges $\mathcal{E}$ indicates direct dependencies between these variables. Let $\mathrm{Pa}(X_j)$ denote the set of parent nodes of node V_j in the graph, and the dependency between variables can be quantified by the conditional probability distribution $P(X_j \mid \mathrm{Pa}(X_j))$. The joint probability distribution represented by the BN can be factorized into the product of local conditional probabilities:

$$P(X_1, X_2, \ldots, X_n) = \prod_{j=1}^{n} P(X_j \mid \mathrm{Pa}(X_j)) \tag{1}$$

where n is the number of nodes in the network. Since the number of valid DAGs grows super-exponentially with n, BNSL has been proven to be an NP-hard problem [29]. In this paper, we use an adjacency matrix to represent candidate structures, which is a common encoding method in GA, as follows:

$$A_{ij} = \begin{cases} 1, & \text{if } X_i \text{ is a parent of } X_j \\ 0, & \text{otherwise} \end{cases} \tag{2}$$

where the adjacency matrix A_{ij} is an $n \times n$ Boolean matrix.

2.2. Mutual information

MI is an important concept in information theory that measures the statistical dependence, or degree of information sharing, between two random variables. Let X and Y be two discrete random variables, and their MI is defined as:

$$MI(X;Y) = \sum_{x\in X}\sum_{y\in Y} p(x,y)\log\frac{p(x,y)}{p(x)p(y)} \tag{3}$$

where $p(x,y)$ is the joint probability distribution of X and Y. $p(x)$ and $p(y)$ are the marginal probability distributions of X and Y.

It is important to note that the standard MI defined in Eq. ((3)) ranges from $(0,+\infty)$. To utilize MI as a probability threshold for the mutation operator, we employ normalized MI (NMI) to map the dependency strength into the interval $[0,1]$. The NMI is defined as:

$$MI_{\text{norm}}(X;Y) = \frac{I(X;Y)}{\min(H(X),H(Y))} \tag{4}$$

where $H(X)$ and $H(Y)$ are the entropies of variables X and Y, respectively. In the remainder of this paper, all references to MI values denote the normalized value $MI_{norm}(X_i;X_j)$.

MI has been applied in various stages of BNSL. Cheng et al. [30] use MI and conditional MI to quantify node dependencies, combined with the Chow-Liu algorithm to generate an initial structure, and then construct a complete network using heuristic EdgeNeeded_H and exact EdgeNeeded tests. Pernkopf et al. [31] propose a sequential greedy heuristic based on conditional MI and classification rates to learn discriminative BN structures efficiently. Campos [32] propose a scoring function based on MI and conditional independence tests, which quantifies the dependency between variables and their parents and combines a penalty term to balance network complexity and data fit. In hybrid learning methods based on meta-heuristic search, MI can constrain the search space in the CB phase [33,34], or guide the search direction of evolutionary algorithms [26].

3. Related work and motivation

This section reviews existing hybrid BNSL methods and highlights the limitations that motivate our proposed method.

3.1. Hybrid Bayesian network structure learning

In CB methods, CI tests are the core for identifying causal skeleton. For discrete BNs, common CI tests include the Likelihood-Ratio Test (LRT) and Pearson's chi-square test. Under the null hypothesis $H_0 : X \perp\!\!\!\perp Y \mid S$, the test statistic ($G^2$ or χ^2_{stat}) follows a chi-square distribution with degrees of freedom $(r_X-1)(r_Y-1)\prod_{i=1}^{|S|}|S_i|$. The resulting p-value is:

$$p = P(\chi^2_{df} > \text{observed statistic}) \tag{5}$$

where p represents the probability of observing the current or more extreme statistics under the null hypothesis. A pre-specified significance level α determines whether to reject H_0: if $p<\alpha$, the edge is retained; otherwise, it is pruned.

SS methods evaluate candidate structures using a scoring function $S(G,D)$. The objective is formalized as $\hat{G} = \arg\max_{G\in\mathcal{G}} S(G,D)$. Common scoring functions include Bayesian functions (e.g., K2 [35], BDeu [36]) and information-theoretic functions (e.g., BIC [37], MDL [38], MIT [32]). Because BIC provides a decomposable criterion that balances data fitting and model complexity and has shown robust empirical performance under finite sample sizes [39], it is used as the scoring function in the SS phase of this work, as defined by the following expression:

$$\text{BIC}(G:D) = \sum_{i=1}^{n}\left(\sum_{j=1}^{q_i}\sum_{k=1}^{r_i} N_{ijk}\log\frac{N_{ijk}}{N_{ij}}\right) - \frac{\log N}{2}\sum_{i=1}^{n} q_i(r_i-1) \tag{6}$$

where n is the number of nodes, q_i is the number of parent combinations for node X_i, and r_i is the number of possible values for node X_i. N_{ijk} is the number of samples in D where node X_i takes the k-th value and its parent combination takes the j-th value. N_{ij} is the number of samples in D where node X_i has the j-th parent combination.

Due to the explosive growth of the DAG search space, SS methods commonly rely on heuristic search strategies. The Greedy Thick Thinning (GTT) [40] method first greedily adds candidate edges to improve the learned structure and then removes weakly supported edges. Following a similar thickening-and-thinning strategy, the three-phase dependency analysis method [30] constructs BN structures by analyzing variable dependencies based on conditional MI and CI tests. Bayesian Search (BS) [35] evaluates candidate structures using posterior probabilities or marginal-likelihood scores and guides local structural modifications according to these scores. Greedy Equivalence Search (GES) [41] improves search efficiency by operating on Markov equivalence classes instead of individual DAGs, and its variants have continued to be used in recent structure learning studies [42].

MMHC [23] first uses the Max-Min Parents and Children (MMPC) method [10] to construct the superstructure, then applies tabu hill climbing to optimize the structure. Constrained optimal search [43] increases the significance level of MMPC to improve the chance of finding the global optimum, though at the cost of increased computational complexity. Adaptively restricted GES [44] combines MMPC with GES [41] and introduces an adaptive shielding-edge strategy to improve model flexibility and consistency for high-dimensional data. Saiyan [45] is an associational heuristic that uses MMD-based dependency scores and graph modifications

to learn a connected DAG for full evidence propagation. Furthermore, SaiyanH [24] extends Saiyan into a hybrid structure learning method by combining MMD-based associational skeleton construction, edge orientation based on CI tests, and BIC-based local refinement.

However, traditional hybrid methods are computationally efficient but prone to local optima. To overcome this limitation, EAs have been introduced into the hybrid framework to enhance global exploration. Ant colony optimization is used to replace HC of the MMHC framework to determine the edge direction within the superstructure [46]. Similarly, Particle Swarm Optimization (PSO) [47] has been combined with the PC algorithm, which incorporates structural prior knowledge and uniform mutation operators. Memetic methods [48] further combine EAs with local heuristic search, where evolutionary operators perform global DAG-structure exploration and heuristic search further refines local structures.

Among the EA-based BNSL methods, GAs have emerged as the dominant search strategy due to their population-based mechanism, which naturally supports the exploration of DAG structures. Contaldi et al. [49] proposed an adaptive evolutionary method that adjusts operator probabilities and maximum parent-set sizes. José et al. [25] employed chi-square CI tests to construct a superstructure and then used the GA to evolve DAGs within the pruned search space. Yan et al. [26] introduced an MI-guided GA that leverages mutual information during initialization to impose effective constraints and guide crossover and cycle-removal operations, thereby accelerating convergence and improving accuracy. Most recently, João et al. [19] adapted the multi-agent GA originally designed for high-dimensional numerical optimization to the combinatorial BNSL task, demonstrating statistical superiority over state-of-the-art methods.

3.2. Motivation

Although hybrid methods offer effective solutions, the upper bound of the search results is strictly determined by the accuracy of the search space restrictions. Traditional approaches typically employ a “restrict-then-search" process with fixed constraints. The use of fixed significance levels narrows the search space while simultaneously limiting the potential to discover the optimal structure. A candidate edge mistakenly pruned during the CB phase, which may correspond to a true causal dependency under appropriate assumptions such as faithfulness and causal sufficiency, cannot be corrected during the subsequent search phase. The fact motivates the development of a dynamic constraint capable of recovering the mistakenly excluded candidate edges.

In the search phase, the fixed parameters and random perturbations in existing EA-based hybrid methods cannot detect the evolutionary state, leading to reduced population diversity and premature convergence. Particularly as the search space expands, the blindness of search operators makes the search process more prone to local optima. Therefore, a search operator that can adapt to dynamic constraints is needed to support effective exploration of potential causal relationships.

Furthermore, EAs require iterative assessment of candidate structures. The frequent structure evaluations in EAs lead to significant computational overhead, and redundant calculations of parent-set scores further restrict algorithm efficiency. The computational burden motivates the construction of an effective score memoization scheme to eliminate redundant evaluations and accelerate structure learning.

4. The proposed method

4.1. Framework overview

To address the limitations of existing hybrid BNSL methods, this paper proposes the CS-CGA that designs hybrid BNSL as a collaborative optimization process. The overall framework of CS-CGA is shown in Fig. 1, which consists of three core modules: the PCR mechanism, the EEAM operator, and the HSM strategy. The core modules are marked with dashed lines in different colors. The PCR mechanism involves constructing the high-confidence superstructure, updating the significance level, and relaxing the superstructure (marked in yellow). The EEAM operator is indicated in red, while the purple module represents the improved HSM strategy. In the framework, each individual within the population represents a candidate DAG structure, encoded as an adjacency matrix.

Before the evolutionary process, CS-CGA employs a strict significance level $\alpha^{(1)}$ for CI tests to construct a high-confidence superstructure composed of reliable dependencies. Then, the edges of initial individuals are generated from the high-confidence superstructure using a random MI threshold and assigned random directions. During the initialization, the loops in individuals are identified and broken by removing the edge with the lowest MI score. Subsequently, the fitness of each individual is evaluated, employing the HSM strategy to accelerate calculation through global topology hashing and local parent-set caching. The best individual is updated based on fitness scores to retain the highest-scoring network discovered.

During the evolutionary phase, the significance level $\alpha^{(t)}$ used in CI tests is adaptively adjusted based on evolutionary state, and the independence constraints are dynamically relaxed to admit additional candidate edges into the relaxed superstructure. Binary tournament selection and parent-set crossover are employed to generate more potential offspring. Next, the EEAM operator is applied to perturb individuals under MI guidance, where each new individual primarily mutates within the high-confidence superstructure to enhance local exploitation. In contrast, duplicate individuals are identified and subjected to a second mutation stage in the relaxed superstructure to help the algorithm escape local optima. Following the genetic operators, the new population is subjected to loop removal and evaluated for fitness. The evolutionary cycle repeats until the preset maximum runtime or iteration number is reached, at which point the current best network structure is output as the final result.

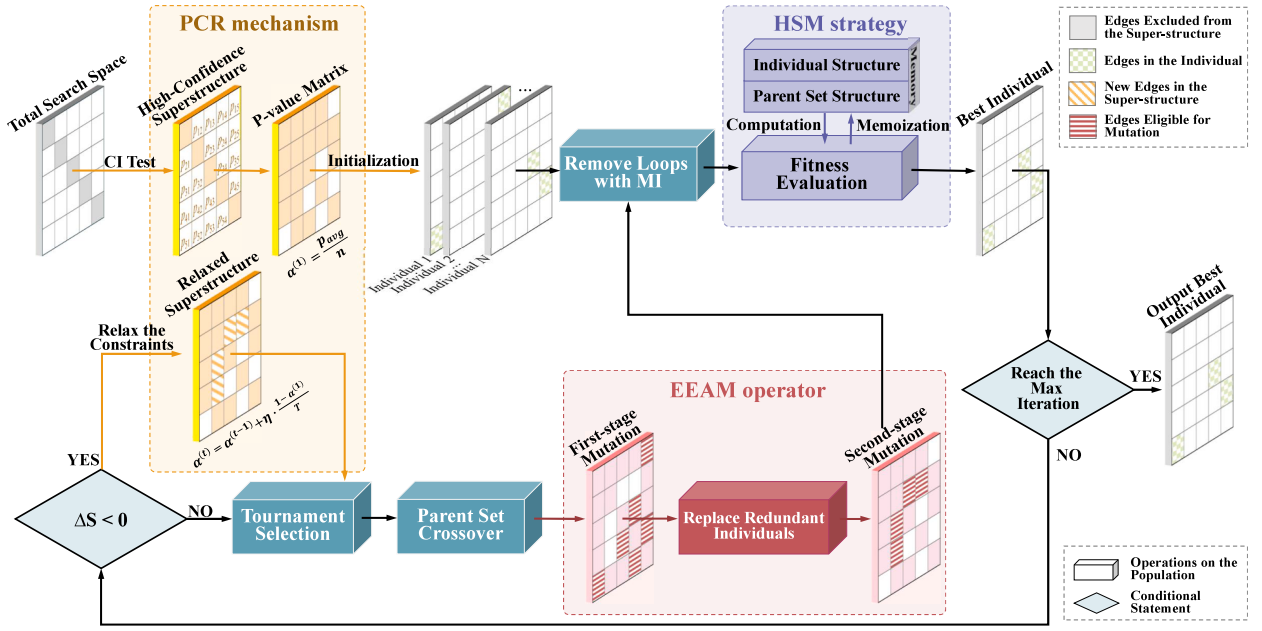


Fig. 1. Overall framework of CS-CGA.

4.2. Progressive constraint relaxation (PCR)

Traditional hybrid BNSL methods typically rely on a fixed significance level α to construct a fixed superstructure during the CB phase, which limits the ability of the subsequent search to explore potential causal relationships. To overcome this limitation, the PCR mechanism dynamically relaxes the constraints of the CB phase when the current population fails to produce a candidate structure comparable to the historical best. The subsequent population evolves within the relaxed superstructure. The collaboration between the CB and SS phases progressively expands the GA search space, thereby enabling the exploration of more potential structures.

The PCR mechanism process begins by constructing a high-confidence superstructure to improve the edge precision of the initial individuals. Specifically, the p-value for each node pair is derived from the calculated G^2 statistic of the LRT. If $p > \alpha$, the nodes are considered conditionally independent, and the corresponding edge is temporarily pruned. The initial significance level $\alpha^{(1)}$ is set to a stringent threshold, defined as follows:

$$\alpha^{(1)} = \frac{p_{\text{avg}}}{n} \tag{7}$$

where n is the total number of variables, and p_{avg} represents the average p-value across all pairwise independence tests.

During evolution, $\alpha^{(1)}$ is relaxed based on an exploration intensity factor η that reflects the normalized score gap between the current best individual S_{current} and the historical best individual S_{best}. The exploration intensity factor is defined as:

$$\eta = \begin{cases} \left[1 + \exp\left(\dfrac{S_{\text{current}} - S_{\text{best}}}{S^{(t)}_{\max} - S^{(t)}_{\min} + \epsilon}\right)\right]^{-1}, & \text{if } S_{\text{current}} < S_{\text{best}}, \\ 0, & \text{otherwise.} \end{cases} \tag{8}$$

where $S^{(t)}_{\max}$ and $S^{(t)}_{\min}$ are the highest and lowest scores in the current population, respectively, and ϵ is a small positive constant. The score gap is rescaled with respect to the current population score range to reduce the influence of different score scales across datasets and scoring functions. Then, the sigmoid function maps the normalized score gap to a bounded and smooth relaxation intensity.

If the current generated population reaches or improves upon the historical best score, the α remains unchanged. Otherwise, α is increased to introduce more potential edges into the superstructure as follows:

$$\alpha^{(t)} = \alpha^{(t-1)} + \eta \cdot \frac{1 - \alpha^{(1)}}{T} \tag{9}$$

where t is the current generation and T is the total number of generations. The ratio $(1 - \alpha^{(1)})/T$ represents the maximum step size for expanding the superstructure, ensuring $\alpha^{(t)} \leq 1$. The PCR mechanism relaxes the CI test threshold only when the current population fails to discover a better structure, while the maximum step size constraint ensures a gradual expansion of the search space.

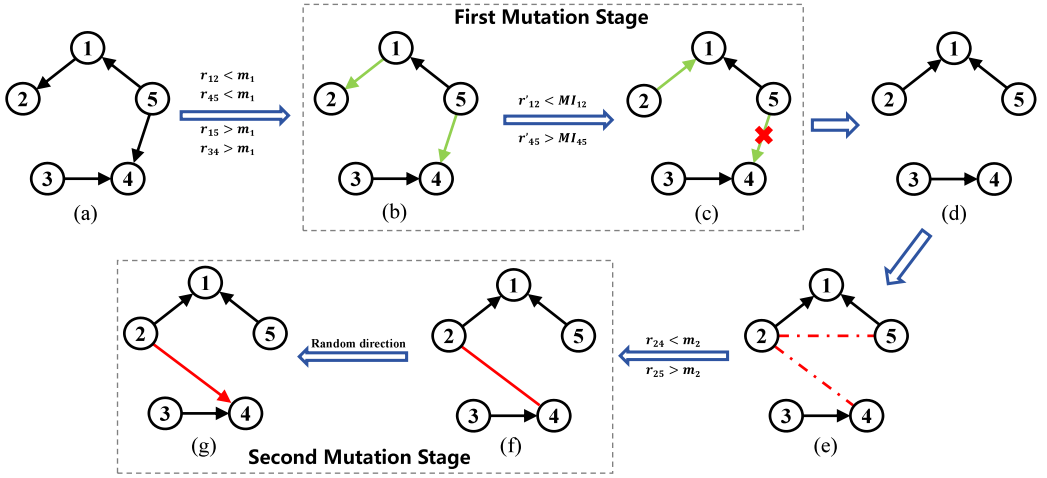


Fig. 2. Illustration of the exploration-enhanced adaptive mutation operator.

4.3. Exploration-enhanced adaptive mutation operator

Most GA-based BNSL methods ignore node dependencies during mutation, leading to convergence to local optima in later stages of evolution. Moreover, the accumulation of duplicate individuals reduces population diversity and search efficiency. To enhance global exploration while preserving local convergence, we propose the EEAM operator with MI-guided mutation and adaptive mutation rate control. As illustrated in Fig. 2, the mutation operator differentiates the mutation process into two stages: the first mutation in the high-confidence superstructure for exploitation and the second mutation in the relaxed superstructure for exploration.

To balance convergence and diversity, the first stage adaptively adjusts the mutation rate in response to the evolutionary state. For each pair of nodes in the high-confidence superstructure, a dynamic mutation rate m_1 is adjusted based on the evolutionary progress and search stagnation degree:

$$m_1 = (1 - \lambda + \eta) \cdot \frac{1}{l_{cnt}} \tag{10}$$

where l_{cnt} is the number of edges in the high-confidence superstructure. The base mutation rate $1/l_{cnt}$ sets the expected number of mutated edges per individual to one, so that the perturbation strength is independent of the superstructure size. The factor $\lambda = t/T$ represents the evolutionary progress (the ratio of the current generation t to the total number of generations T), and gradually reduces the mutation rate as evolution proceeds to promote convergence. When the current optimal structure fails to surpass the global best, the adaptive factor η dynamically increases the mutation rate to explore more potential structures.

The second stage is designed to escape local optima by leveraging the relaxed superstructure. A duplication analysis is first conducted to identify individuals that exist in both the current and historical populations. Then, a portion of these duplicates is replaced by the historical best individual, with the replacement ratio increasing linearly from 0 to 0.5 over generations. Subsequently, the remaining duplicates and the replaced individuals undergo a second mutation in the edges recovered by the PCR mechanism. The mutation rate m_2 in the second stage is defined as:

$$m_2 = \frac{1}{l_{ext}} \tag{11}$$

where l_{ext} is the total number of recovered edges, the inverse formulation of m_2 and l_{ext} maintains the expectation that each individual mutates approximately one recovered edge on average, thus enabling stable exploration within the expanded search space.

Fig. 2 illustrates the mutation process using a five-node network corresponding to the adjacency matrix in Fig. 1. In the first stage, a random number r is generated for each node pair within the high-confidence superstructure. If this random number is less than the mutation threshold m_1 (e.g., assuming $r_{12} < m_1$ and $r_{45} < m_1$ in Fig. 2), the mutation operation is triggered for that pair. Similarly, within recovered edges of the relaxed superstructure, random numbers are generated for the specific node pairs (e.g., $(2, 4)$ and $(2, 5)$). As shown in the example, since $r_{24} < m_2$, the mutation operation is applied to the node pair $(2, 4)$.

Once a node pair is selected for mutation in either stage, the MI-guided mutation strategy is executed as follows:

- If there is a directed edge between the nodes, another random number $r'_{ij} \in [0, 1]$ is generated. If the corresponding $MI(i, j) > r'_{ij}$, the direction of the edge is reversed (e.g., nodes $1 \rightarrow 2$ becomes nodes $2 \rightarrow 1$ in Fig. 2(c)). Otherwise, the edge is removed (e.g., nodes $5 \rightarrow 4$ is removed in Fig. 2(c)).
- If there is no edge between the two nodes, a directed edge is added with equal probability in either direction (e.g., nodes $2 \rightarrow 4$ in Fig. 2(g)).

4.4. Hierarchical score memoization with global topology hashing and local parent-set caching

4.4.1. The proposed HSM strategy

The fitness evaluation is the most time-consuming part of hybrid methods, so the efficiency of scoring function calculation is a decisive factor in overall performance. This paper proposes an HSM strategy as outlined in Algorithm 1. The HSM strategy enables efficient score reuse across the entire network and the parent-set structures. Due to the randomness of evolutionary operators, the same network structure may recur across generations. Lines 1 to 4 of the pseudocode implement this efficient retrieval. Each network structure G is mapped to a unique identifier via the canonical encoding of its edge set:

$$\mathcal{E}(G) = \{(j,k) \mid G_{jk} = 1\} \tag{12}$$

where edges are serialized lexicographically into a string key to query the global hash table $\mathcal{H}_{\text{global}}$. A hit at this level instantly returns the total score, bypassing all node-level computations.

Furthermore, even if the overall structure differs, many individuals may share part of the parent-set structures for certain nodes. To alleviate the burden of evaluating the repeated local substructures, lines 6 to 15 introduce a local parent-set caching mechanism. For each node X_i, a dynamic cache C_i stores historical parent-set structures $\text{Pa}(X_i)$ and corresponding local scores. Given that parent-set structures tend to converge in the later stages of evolution, cyclic overwriting is employed to manage memory in the dynamic search.

Algorithm 1 Hierarchical score memoization strategy.

Require: Network structure G, dataset D, global hash table $\mathcal{H}_{\text{global}}$, node-level caches $\{C_i\}_{i=1}^{n}$
Ensure: Structure score $\text{Score}(G)$
1: key ← canonical_encode(G) *// Encode the whole topology*
2: **if** key ∈ $\mathcal{H}_{\text{global}}$ **then**
3: **return** $\mathcal{H}_{\text{global}}$[key] *// Reuse the complete DAG score*
4: **end if**
5: score ← 0
6: **for** each node X_i in G **do**
7: pa ← $\text{Pa}(X_i)$
8: m ← bitmask(pa) *// Encode the parent set*
9: **if** $m \in C_i$.masks **then**
10: local_score ← C_i.scores[index(m)] *// Reuse a cached local score*
11: **else**
12: local_score ← compute_local_score(X_i, pa, D)
13: C_i.update(m, local_score) *// Store the new parent-set score*
14: **end if**
15: score ← score + local_score
16: **end for**
17: $\mathcal{H}_{\text{global}}$[key] ← score *// Cache the complete DAG score*
18: **return** score

4.4.2. Acceleration efficiency analysis

Since fitness evaluation typically consumes the vast majority of computational resources in evolutionary structure learning, this section focuses on the time complexity analysis of the scoring process rather than the global algorithmic complexity.

Topology hashing reduces the lookup complexity for a recurring individual from $O(M)$ (where M is the cumulative number of generated individuals across all generations) to $O(1)$ on average. The parent-set cache requires counting the joint states of the parents of node X_i, denoted Pi_i. The time complexity for evaluating the structure of an individual from the dataset D is:

$$T_{\text{base}} = O\left(|D| \sum_{i=1}^{n} |\Pi_i|\right) \tag{13}$$

where $|\Pi_i|$ represents the number of parent-set states for node i. The number of nodes is n, and $|D|$ is the number of samples in the dataset. When dealing with large parent sets or high data variance, $|\Pi_i|$ can grow exponentially.

While the HSM strategy does not alter the worst-case complexity of a completely novel structure, it can reduce the expected runtime through the probabilistic collaboration of the two cache levels. Let the average hit rates for the global topology hash and the local parent-set cache be h_{global} and h_{local}, respectively. The expected actual scoring time T_{actual} is approximated by:

$$T_{\text{actual}} \approx (1 - h_{\text{global}}) \cdot (1 - h_{\text{local}}) \cdot T_{\text{base}} \tag{14}$$

As the population converges, both h_{global} and h_{local} increase markedly, resulting in $T_{\text{actual}} \ll T_{\text{base}}$. Furthermore, the acceleration efficiency of the HSM strategy applies to all locally additive scoring functions, allowing for integration into existing SS and hybrid learning methods.

Table 1
Characteristics of benchmark BNs and sample sizes.

BN	Nodes	Edges	Max in-degree	Free Parameters	Datasets
Asia	8	8	2	18	AS-500 AS-1000 AS-3000
Insurance	27	52	3	984	INS-500 IN-1000 IN-3000
Water	32	66	4	10083	WA-500 WA-1000 WA-3000
Alarm	37	46	4	509	AL-500 AL-1000 AL-3000
Hailfinder	56	66	4	2656	HA-500 HA-1000 HA-3000
Hepar II	70	123	6	1453	HE-500 HE-1000 HE-3000
Win95pts	76	112	7	574	WI-500 WI-1000 WI-3000
Andes	223	338	6	1157	AN-500 AN-1000 AN-3000

5. Experiment and analysis

This section aims to evaluate the performance of the proposed CS-CGA and validate the effectiveness of the PCR mechanism, the EEAM operator, and the HSM strategy. Experiments are conducted on eight benchmark BNs [9], namely Asia, Insurance, Water, Alarm, Hailfinder, Hepar II, Win95pts, and Andes. Performance comparisons are carried out against six representative evolutionary BNSL methods, including PSX [28], EKGA-BN [22], AESL-GA [49], hybrid-SLA [25], MIGA [26], and MAGABN [19]. Furthermore, two classical SS methods (GTT [40] and BS [35]), the recently proposed SS method IGES [42], and the established hybrid method saiyanH [24] are employed as additional comparison methods to evaluate CS-CGA against a broader range of BNSL methods.

5.1. Experimental setup

The detailed characteristics of the benchmark BNs are summarized in Table 1. These networks span a wide range of topological complexities, with node counts ranging from 8 to 223. For each BN, datasets containing 500, 1000, and 3000 samples are generated independently. Ten datasets are generated for each sample size, yielding a total of 240 test datasets.

CS-CGA, the GA-based methods, and the additional comparison methods are implemented in MATLAB R2021a using the Bayes Net Toolbox (BNT) [50]. The compared methods use the built-in BNT cache to reuse local BIC scores, whereas CS-CGA replaces the BNT cache with the proposed HSM strategy. The experimental platform is equipped with an AMD R7-7840HS processor and 64.0 GB of RAM, running Windows 10.

To ensure fair comparisons, all GA-based algorithms use the same population size and generation budget. The population size is set to 100, and the maximum number of generations is set to 200. The BIC score is used as the structure score for all methods. For the compared hybrid algorithms that include a fixed constraint-based stage, CI tests are performed using the LRT with a fixed significance level of $\alpha = 0.01$. In CS-CGA, the initial threshold of the PCR mechanism is computed from the data according to Eq. (7). The threshold is then progressively relaxed during evolution according to the PCR update rule.

Four core metrics are used to assess algorithmic performance comprehensively:

- F1 score: A harmonic measure of precision and recall, defined as

$$F1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}, \tag{15}$$

  where precision denotes the proportion of correctly identified edges among all predicted edges, and recall denotes the proportion of true edges that are correctly recovered. A higher F1 score indicates more accurate structural recovery with fewer edge errors.
- Structural Hamming Distance (SHD): The total number of edge additions, deletions, and direction reversals required to transform the learned DAG into the actual structure. Lower SHD values indicate closer alignment with the ground truth.

- BIC score: Used to plot convergence curves and evaluate search efficiency and convergence behavior.
- Runtime: The total execution time in seconds.

5.2. Performance analysis of CS-CGA

5.2.1. Structural accuracy

The structural accuracy of CS-CGA is compared with that of representative evolutionary, non-EA SS, and non-EA hybrid methods using F1 score and SHD on eight benchmark BNs. As shown in Tables 2 and 3, CS-CGA achieves the highest F1 scores and the lowest SHD values on most of the 24 datasets. On the Asia, Insurance, Alarm, and Hepar II datasets, CS-CGA obtains the best F1 scores and the lowest SHD values among all methods that complete the experiments. CS-CGA also obtains the best F1 scores on all Hailfinder datasets and the lowest SHD values on HA-1000 and HA-3000. On the high-dimensional Andes network, CS-CGA remains the best-performing method on the corresponding datasets. The F1 and SHD advantages indicate that CS-CGA improves the recovery of benchmark structural edges while controlling overall structural errors.

On the WA-1000 and WA-3000 datasets, MIGA obtains slightly higher mean F1 scores, whereas CS-CGA achieves lower SHD values on all Water datasets. The Water network contains multivalued variables and complex high-order CPTs, for which the MI-guided crossover operator of MIGA preserves more strongly dependent candidate edges. However, the crossover of MIGA biases newly generated individuals toward candidate edges with high MI values, thereby overfitting the MI values derived from the observed samples. The F1 score of MIGA on IN-3000 is 12.38 percentage points lower than that of CS-CGA. On the Alarm datasets, the F1 scores of MIGA are 8.30, 6.73, and 8.00 percentage points lower than those of CS-CGA, respectively. IGES performs better on some Win95pts datasets, as its equivalence-class search and CMI-based filtering contribute to structure learning on sparse binary networks, although the F1 and SHD results are inconsistent across WI-500, WI-1000, and WI-3000.

Overall, CS-CGA exhibits robust structural accuracy in terms of F1 score and SHD across the majority of benchmark datasets. The results indicate that CS-CGA enhances the identification of benchmark structural edges, benefiting from the collaborative loop established by the PCR mechanism and the search capability of the EEAM operator.

5.2.2. Runtime analysis

Table 4 compares the runtime of CS-CGA with the GA-based and the non-EA methods. CS-CGA achieves lower runtime than all GA-based methods across the 24 datasets. The non-EA methods are faster on the smaller Asia, Insurance, and Water networks. GTT achieves the lowest runtime on the Asia datasets, while saiyanH is faster on the Insurance and Water datasets. In contrast, CS-CGA achieves the lowest runtime on all Alarm, Hailfinder, Hepar II, Win95pts, and Andes datasets among the methods that complete the experiments. GTT, BS, and IGES time out on all three Andes datasets.

The speed advantages of the non-EA methods over smaller BNs stem from their deterministic or greedy search procedures. Asia contains only eight nodes and eight edges, so GTT and BS can construct and refine a structure through a small number of edge operations, while saiyanH can quickly obtain a restricted skeleton before BIC-based local refinement. As the number of variables and candidate edges increases, GTT and BS repeatedly assess an expanding set of candidate additions, deletions, and local modifications to edges. IGES additionally evaluates equivalence-class operations and performs CMI-based filtering, requiring 3510.5 and 5714.2 seconds on WI-500 and WI-3000, respectively, before timing out on all Andes datasets. The skeleton construction and BIC-based local refinement of saiyanH become increasingly costly. On the 223-node Andes network, saiyanH requires 1021.3, 1816.8, and 4923.9 seconds, which are approximately 3.0, 4.9, and 10.0 times the runtime of CS-CGA on AN-500, AN-1000, and AN-3000, respectively.

The runtime advantage of CS-CGA becomes more apparent on networks with larger search spaces. On the WI-3000 dataset, CS-CGA completes learning in 121.3 seconds, which is approximately 21 and 29 times faster than PSX and AESL-GA, respectively. On the AN-500 dataset, CS-CGA requires 336.1 seconds, which is 3.4% of the runtime of MIGA and less than one-third of that of saiyanH. On the AN-3000 dataset, CS-CGA completes the search in 491.9 seconds, whereas GTT, BS, and IGES time out, saiyanH requires 4923.9 seconds, and all GA-based methods require more than 16,000 seconds.

The results validate the effectiveness of the proposed HSM strategy in CS-CGA. By integrating global topology hashing with local parent-set caching, redundant score evaluations are reduced. Particularly in high-dimensional BNs, CS-CGA improves runtime efficiency while maintaining high structure-learning accuracy.

5.2.3. Statistical significance analysis

To assess whether the observed performance differences are statistically significant, two-sided Wilcoxon signed-rank tests are conducted separately on the ten matched runs for each completed dataset and sample-size setting. F1 score, SHD, and runtime are tested separately for CS-CGA against every compared method. Methods completing all experiments are evaluated on 24 settings, whereas GTT, BS, and IGES are evaluated on 21 settings because they time out on the Andes datasets.

As shown in Table 5, for each metric, the p-values for the compared methods in the same setting are adjusted using the Holm correction, and a Holm-adjusted p-value < 0.05 is considered statistically significant. W, NS, and L denote the numbers of completed settings in which CS-CGA performs significantly better, shows no significant difference, and performs significantly worse than the compared method, respectively. CS-CGA obtains significant wins in 12 to 22 completed settings for F1 score and in 12 to 23 settings for SHD against each compared method. No significant loss is observed for F1 score, and only one significant SHD loss is observed against IGES.

In terms of runtime, CS-CGA achieves significant gains across all 24 settings compared to PSX, EKGA-BN, AESL-GA, hybrid-SLA, and MIGA. Against MAGABN, CS-CGA obtains significant wins in 19 of the 24 settings. The runtime comparisons with the non-EA

Table 2
F1 scores (%) of CS-CGA and representative evolutionary, non-EA SS, and non-EA hybrid methods (Mean ± Std).

Datasets	CS-CGA	MIGA	hybrid_SLA	EKGA	AESL-GA	PSX	MAGABN	GTT	BS	IGES	saiyanH
AS-500	**90.22%(6.17)**	89.46%(6.21)	89.63%(5.78)	89.46%(6.21)	89.46%(6.21)	89.51%(6.24)	**90.22%(6.17)**	86.07%(7.84)	84.41%(7.20)	74.94%(3.59)	75.52%(9.21)
AS-1000	**91.10%(5.93)**	**91.10%(5.93)**	**91.10%(5.93)**	**91.10%(5.93)**	90.95%(6.20)	**91.10%(5.93)**	90.24%(7.79)	83.32%(8.17)	83.98%(10.50)	81.92%(4.99)	86.50%(8.44)
AS-3000	**95.14%(3.19)**	93.19%(7.31)	93.10%(5.17)	93.19%(7.31)	93.14%(7.31)	93.19%(7.31)	**95.14%(3.19)**	82.22%(12.39)	81.63%(8.57)	79.23%(3.52)	84.41%(7.54)
IN-500	**70.71%(2.61)**	63.51%(5.37)	62.05%(5.97)	57.06%(4.12)	59.07%(3.49)	56.21%(6.69)	56.11%(5.16)	62.15%(3.03)	60.73%(3.58)	56.20%(1.45)	66.25%(1.71)
IN-1000	**75.93%(3.18)**	69.76%(1.66)	69.46%(5.31)	67.15%(4.48)	64.51%(4.59)	58.84%(5.70)	59.34%(5.90)	66.84%(4.42)	67.13%(3.78)	62.79%(2.13)	71.89%(3.44)
IN-3000	**87.33%(1.28)**	74.95%(3.00)	76.01%(4.54)	76.11%(3.20)	68.82%(4.96)	61.82%(3.07)	69.52%(5.89)	77.10%(3.66)	76.13%(4.37)	72.17%(1.04)	75.62%(3.17)
WA-500	**44.42%(1.13)**	42.67%(1.69)	40.82%(3.10)	38.97%(2.47)	39.14%(3.25)	40.29%(2.37)	38.18%(2.23)	43.42%(2.80)	42.74%(1.79)	36.59%(0.83)	41.13%(1.81)
WA-1000	44.95%(2.28)	**45.89%(2.01)**	42.72%(2.14)	40.17%(2.91)	41.73%(2.65)	44.52%(3.20)	42.72%(2.13)	43.36%(2.83)	43.95%(2.64)	35.20%(0.52)	42.54%(1.59)
WA-3000	52.30%(2.14)	**53.64%(2.16)**	51.91%(2.11)	48.13%(2.47)	51.17%(2.65)	49.74%(2.22)	46.58%(3.65)	51.56%(2.20)	52.34%(2.10)	46.00%(2.04)	48.83%(3.18)
AL-500	**83.43%(4.14)**	75.13%(4.56)	63.60%(6.68)	64.24%(10.56)	45.80%(7.62)	51.09%(4.99)	57.38%(7.57)	78.96%(3.77)	79.29%(3.69)	74.43%(3.59)	71.52%(4.50)
AL-1000	**87.43%(3.24)**	80.70%(4.28)	66.31%(4.94)	73.41%(8.58)	45.81%(4.38)	54.72%(6.68)	54.40%(5.66)	81.36%(2.80)	77.39%(4.17)	77.05%(2.40)	73.38%(4.22)
AL-3000	**91.51%(3.32)**	83.51%(3.39)	68.43%(6.70)	78.63%(7.75)	48.78%(6.74)	57.13%(6.58)	59.24%(5.53)	83.31%(4.29)	85.06%(4.09)	82.13%(1.33)	81.78%(5.42)
HA-500	**60.40%(5.61)**	58.91%(4.04)	52.69%(3.81)	54.59%(1.54)	42.36%(4.43)	42.85%(5.12)	47.44%(2.98)	59.50%(4.57)	52.23%(4.11)	59.56%(2.66)	47.39%(2.42)
HA-1000	**69.23%(5.18)**	61.64%(5.23)	51.28%(5.37)	59.33%(2.87)	40.51%(4.53)	46.04%(8.19)	50.48%(5.33)	66.58%(2.69)	60.35%(3.97)	68.44%(3.21)	51.95%(1.89)
HA-3000	**73.30%(3.97)**	65.76%(3.79)	59.14%(4.93)	64.63%(3.22)	38.48%(4.11)	47.38%(1.85)	52.96%(5.98)	67.28%(4.73)	62.28%(5.40)	70.20%(2.18)	55.97%(1.88)
HE-500	**47.10%(1.49)**	44.22%(2.03)	43.07%(2.40)	41.85%(2.31)	40.88%(2.30)	43.07%(1.87)	37.36%(2.90)	36.75%(4.92)	36.06%(6.56)	42.12%(2.81)	40.21%(3.23)
HE-1000	**56.03%(1.65)**	53.31%(2.74)	52.28%(1.79)	50.25%(2.18)	48.59%(2.51)	48.78%(2.65)	44.47%(2.22)	47.39%(2.38)	45.14%(2.80)	49.87%(1.73)	48.39%(2.09)
HE-3000	**70.33%(1.58)**	66.47%(2.08)	66.65%(2.88)	67.04%(2.51)	45.39%(2.00)	54.21%(1.99)	56.42%(1.89)	58.86%(2.16)	58.39%(1.82)	62.28%(1.82)	64.96%(1.38)
WI-500	62.00%(4.31)	57.91%(3.19)	50.90%(4.94)	44.46%(3.76)	46.64%(4.01)	47.07%(5.14)	40.76%(4.86)	54.32%(3.23)	42.19%(7.25)	**62.94%(2.87)**	44.40%(2.99)
WI-1000	**68.71%(2.08)**	63.20%(3.52)	52.28%(3.92)	46.14%(3.69)	47.92%(6.09)	47.77%(2.73)	42.34%(3.26)	60.67%(3.91)	47.88%(5.11)	66.43%(3.26)	49.58%(1.82)
WI-3000	72.33%(2.97)	66.92%(3.59)	55.63%(3.81)	48.17%(3.11)	47.33%(5.39)	46.28%(3.79)	40.95%(4.31)	59.03%(7.72)	54.29%(7.14)	**75.22%(2.65)**	60.73%(2.82)
AN-500	**69.51%(1.18)**	60.21%(1.73)	56.14%(1.97)	54.33%(1.79)	50.07%(2.25)	50.00%(1.40)	40.46%(1.31)	Timeout	Timeout	Timeout	38.62%(1.81)
AN-1000	**71.31%(1.36)**	65.05%(1.39)	55.71%(1.60)	55.50%(1.77)	49.11%(2.95)	43.78%(1.73)	39.43%(2.64)	Timeout	Timeout	Timeout	45.11%(1.57)
AN-3000	**75.44%(1.89)**	71.27%(1.83)	53.07%(1.31)	51.38%(1.73)	43.78%(1.82)	45.88%(2.01)	41.21%(3.26)	Timeout	Timeout	Timeout	54.34%(1.00)

Table 3
SHD values of CS-CGA and representative evolutionary, non-EA SS, and non-EA hybrid methods (Mean ± Std).

Datasets	CS-CGA	MIGA	hybrid_SLA	EKGA	AESL-GA	PSX	MAGABN	GTT	BS	IGES	saiyanH
AS-500	**1.75(1.01)**	1.85(0.98)	1.80(0.84)	1.85(0.98)	1.85(0.98)	1.80(1.01)	**1.75(1.01)**	2.55(1.27)	3.00(1.30)	3.30(0.51)	4.60(1.51)
AS-1000	**1.60(0.77)**	**1.60(0.77)**	**1.60(0.77)**	**1.60(0.77)**	1.65(0.87)	**1.60(0.77)**	1.80(1.23)	3.65(1.75)	3.10(1.83)	2.80(0.33)	2.75(1.10)
AS-3000	**1.10(0.44)**	1.40(1.11)	1.45(0.96)	1.40(1.11)	1.45(1.11)	1.40(1.11)	**1.10(0.44)**	3.80(2.12)	4.15(1.47)	2.85(0.23)	3.00(1.07)
IN-500	**25.30(2.27)**	31.05(4.05)	32.90(4.95)	36.45(3.22)	34.55(2.71)	36.55(5.11)	37.45(3.60)	32.85(2.38)	33.80(2.59)	34.50(1.23)	30.20(1.45)
IN-1000	**22.10(3.07)**	28.20(1.76)	27.85(4.86)	29.65(3.94)	31.45(3.93)	36.70(4.28)	36.45(4.94)	30.80(3.53)	30.45(3.11)	30.55(1.29)	26.25(3.05)
IN-3000	**12.55(1.31)**	25.95(2.68)	24.05(4.51)	23.60(2.84)	29.70(4.48)	37.65(2.78)	30.95(5.73)	24.00(3.44)	25.35(3.94)	26.25(1.03)	24.70(3.04)
WA-500	**48.25(0.84)**	49.50(1.23)	51.10(2.32)	52.50(2.13)	52.45(2.76)	51.95(1.65)	53.35(1.90)	48.85(2.05)	49.30(1.08)	51.80(0.40)	57.40(1.66)
WA-1000	**47.45(1.37)**	48.20(1.58)	50.10(1.84)	52.05(2.39)	50.85(2.06)	49.30(2.57)	50.45(1.54)	49.70(2.36)	48.90(1.85)	52.50(0.22)	56.20(1.40)
WA-3000	**43.20(1.78)**	44.35(2.06)	44.95(1.99)	48.15(4.69)	45.30(2.11)	46.95(2.12)	50.00(4.02)	45.20(1.90)	44.60(1.66)	47.15(1.25)	51.70(2.83)
AL-500	**16.85(3.56)**	24.95(4.11)	33.30(4.87)	32.40(8.46)	46.60(6.20)	42.95(3.58)	38.60(6.39)	22.55(3.32)	22.55(1.97)	21.05(2.38)	24.60(3.88)
AL-1000	**13.35(3.09)**	21.35(4.08)	32.55(4.15)	26.20(7.07)	48.90(3.18)	42.85(5.93)	44.05(4.63)	22.75(1.97)	26.10(4.74)	19.65(2.36)	23.40(3.53)
AL-3000	**10.10(3.51)**	21.25(4.11)	33.95(5.83)	23.25(7.76)	50.90(6.76)	45.60(7.15)	43.85(4.95)	24.05(4.45)	21.75(5.09)	15.70(1.10)	16.40(4.88)
HA-500	45.90(6.18)	48.35(4.18)	55.15(4.01)	53.05(1.64)	64.10(5.33)	65.15(5.23)	59.15(3.12)	**43.90(4.70)**	48.50(3.32)	49.45(3.20)	64.15(2.41)
HA-1000	**36.90(5.95)**	46.50(5.13)	59.05(6.09)	49.55(3.16)	70.75(5.80)	64.55(8.18)	58.30(5.62)	37.10(2.60)	42.50(3.77)	38.45(2.84)	59.05(2.15)
HA-3000	**33.05(4.71)**	42.90(4.45)	53.75(6.05)	45.50(4.56)	78.35(5.89)	70.35(2.83)	60.05(7.53)	38.70(5.00)	42.60(5.48)	36.35(2.25)	54.75(1.93)
HE-500	**90.50(2.72)**	97.00(2.49)	99.95(3.77)	101.30(3.80)	103.10(3.67)	96.20(3.36)	104.65(5.70)	105.70(6.87)	107.10(8.48)	96.85(3.50)	114.40(5.63)
HE-1000	**80.10(2.97)**	85.10(4.38)	87.85(3.31)	90.20(3.05)	91.85(4.45)	90.80(4.70)	96.75(3.35)	91.10(2.70)	94.20(4.04)	86.35(2.58)	100.15(3.79)
HE-3000	**58.45(2.66)**	66.95(2.95)	67.50(5.20)	66.55(4.44)	107.65(3.87)	90.65(3.65)	82.00(3.15)	76.00(3.63)	75.80(3.34)	69.90(2.49)	71.35(2.66)
WI-500	**74.35(8.40)**	85.10(6.09)	96.40(9.31)	115.95(6.85)	103.80(7.70)	105.80(11.99)	116.45(9.94)	85.40(6.02)	105.00(11.35)	82.05(5.96)	140.65(8.49)
WI-1000	64.25(4.19)	78.95(6.27)	101.05(8.89)	120.15(8.49)	107.00(11.30)	111.25(6.57)	121.00(5.62)	77.50(7.83)	99.70(9.98)	**63.55(5.32)**	128.55(4.28)
WI-3000	60.60(6.22)	75.65(7.84)	107.70(10.51)	129.85(10.74)	118.85(11.53)	129.65(9.82)	135.80(9.84)	84.80(15.47)	94.60(15.97)	**48.80(4.55)**	98.65(9.73)
AN-500	**178.15(8.26)**	249.25(10.39)	288.55(13.70)	307.45(13.11)	314.20(16.97)	311.75(9.01)	320.90(8.69)	Timeout	Timeout	Timeout	593.05(31.38)
AN-1000	**181.70(8.11)**	224.40(7.86)	309.15(10.27)	316.60(15.57)	332.10(21.93)	333.65(14.42)	331.20(16.09)	Timeout	Timeout	Timeout	540.65(26.01)
AN-3000	**167.65(14.41)**	193.65(10.61)	365.05(11.21)	373.10(12.97)	384.15(15.49)	392.60(13.72)	340.35(18.40)	Timeout	Timeout	Timeout	443.05(15.51)

Table 4
Runtime (s) of CS-CGA and representative evolutionary, non-EA SS, and non-EA hybrid methods (Mean ± Std).

Datasets	CS-CGA	MIGA	hybrid_SLA	EKGA	AESL-GA	PSX	MAGABN	GTT	BS	IGES	saiyanH
AS-500	6.9(1.4)	18.7(0.2)	20.0(0.2)	35.7(2.2)	27.3(2.8)	26.6(0.3)	26.1(1.2)	**0.6(0.1)**	1.0(0.1)	1.2(0.1)	1.5(0.6)
AS-1000	7.1(1.3)	20.9(0.2)	21.9(0.1)	41.4(4.1)	27.1(0.3)	29.1(0.2)	27.3(1.1)	**0.6(0.1)**	1.0(0.1)	1.1(0.1)	1.1(0.0)
AS-3000	7.2(0.8)	29.0(0.9)	31.6(0.6)	68.3(1.1)	42.8(4.8)	41.7(0.6)	30.1(2.5)	**0.6(0.1)**	1.1(0.1)	1.3(0.1)	1.3(0.0)
IN-500	28.4(6.5)	110.7(2.7)	148.6(6.9)	301.5(40.6)	258.8(20.6)	193.1(7.3)	229.0(15.0)	27.6(1.0)	34.2(5.3)	67.1(4.9)	**20.0(6.3)**
IN-1000	36.5(3.9)	127.1(1.3)	181.8(6.0)	297.0(20.7)	345.5(17.8)	230.5(5.6)	222.5(13.8)	31.9(1.3)	39.3(3.9)	75.2(3.1)	**11.7(3.0)**
IN-3000	65.9(1.6)	178.8(1.8)	268.4(7.9)	446.6(18.5)	464.9(13.8)	327.7(4.2)	258.9(13.2)	40.4(1.3)	66.4(8.0)	96.6(3.4)	**11.2(0.3)**
WA-500	25.5(2.1)	115.9(0.8)	136.5(1.4)	217.2(3.4)	206.2(2.8)	180.6(2.1)	238.5(10.1)	25.9(0.8)	27.6(4.7)	69.2(1.7)	**6.9(1.5)**
WA-1000	29.4(2.8)	135.0(0.9)	157.4(1.2)	253.0(3.4)	242.1(3.2)	210.5(2.2)	272.5(12.7)	28.0(1.2)	36.2(5.1)	73.7(2.6)	**6.3(0.8)**
WA-3000	35.0(2.8)	192.6(1.1)	229.5(2.5)	363.8(6.8)	364.5(14.0)	310.2(5.2)	254.8(10.4)	36.4(1.2)	44.7(2.9)	97.3(2.3)	**11.1(0.6)**
AL-500	**42.6(3.4)**	263.4(20.0)	360.3(6.0)	584.7(5.5)	659.9(18.3)	397.8(6.2)	472.4(30.1)	67.6(3.3)	61.3(5.8)	194.8(9.5)	52.1(11.6)
AL-1000	**49.3(1.8)**	261.7(37.6)	394.8(3.6)	631.8(3.7)	669.9(21.4)	452.8(4.4)	466.6(16.6)	76.8(3.0)	53.0(8.0)	207.0(8.1)	69.4(15.9)
AL-3000	**53.9(3.3)**	304.6(3.0)	484.0(3.0)	743.2(4.6)	861.9(13.8)	578.5(6.3)	486.5(38.5)	96.0(3.6)	71.7(10.4)	220.4(18.0)	82.4(9.7)
HA-500	**60.3(1.4)**	319.6(6.4)	470.2(12.2)	746.5(26.2)	872.7(38.3)	577.5(11.4)	712.7(72.2)	228.2(8.7)	87.1(4.7)	703.3(28.8)	102.9(12.4)
HA-1000	**65.9(1.8)**	361.3(2.3)	577.7(12.1)	909.6(12.3)	1008.0(7.2)	682.8(11.7)	795.9(82.9)	250.2(8.3)	112.6(8.4)	796.5(19.5)	110.3(9.7)
HA-3000	**79.6(0.9)**	901.4(3.8)	1133.8(6.5)	1737.5(8.5)	1796.1(13.4)	1053.2(114.0)	870.3(93.6)	365.7(29.1)	187.9(16.3)	998.9(94.2)	153.3(10.9)
HE-500	**55.5(4.4)**	369.3(8.7)	440.1(26.4)	730.3(26.4)	846.0(97.8)	448.1(11.3)	968.0(68.0)	358.3(23.1)	118.5(11.7)	2148.4(199.4)	169.7(18.3)
HE-1000	**59.8(1.7)**	541.9(83.4)	574.7(19.0)	958.3(47.8)	1164.3(171.6)	558.3(13.2)	1046.8(86.4)	400.9(14.7)	140.7(13.0)	1576.2(95.1)	200.2(12.7)
HE-3000	**75.8(3.2)**	1978.2(28.6)	1396.0(26.3)	2339.6(46.4)	2546.5(346.1)	1266.2(10.8)	1185.8(105.0)	557.1(17.2)	200.4(18.2)	1946.2(73.5)	290.2(19.2)
WI-500	**108.1(4.2)**	635.5(20.6)	1118.0(14.3)	949.0(24.7)	1764.9(40.3)	1336.5(15.9)	1621.3(151.9)	753.4(44.9)	448.7(23.7)	3510.5(974.6)	272.8(20.9)
WI-1000	**110.6(4.7)**	725.7(7.0)	1246.4(12.5)	1074.9(12.0)	2119.1(173.5)	1859.4(37.6)	1892.4(188.4)	870.5(37.6)	588.8(30.4)	5510.7(318.0)	328.9(39.0)
WI-3000	**121.3(2.7)**	1479.1(6.0)	2143.8(57.5)	1822.1(28.4)	3538.0(83.2)	2539.8(42.7)	1983.4(136.6)	1082.9(53.5)	776.3(49.2)	5714.2(960.7)	164.0(9.1)
AN-500	**336.1(21.0)**	9991.7(593.2)	9246.4(312.3)	8131.7(347.0)	16921.7(1565.2)	7302.3(133.5)	15047.7(412.8)	Timeout	Timeout	Timeout	1021.3(8.6)
AN-1000	**373.9(19.4)**	13395.4(1394.0)	13220.8(470.6)	12234.7(668.1)	29540.1(3152.8)	10428.9(87.2)	14054.1(524.6)	Timeout	Timeout	Timeout	1816.8(395.5)
AN-3000	**491.9(43.2)**	16267.7(637.1)	25145.3(595.7)	23507.8(497.4)	40167.3(59.7)	17462.7(48.0)	17286.0(785.8)	Timeout	Timeout	Timeout	4923.9(1307.5)

Table 5
Summary of setting-level Wilcoxon signed-rank tests for CS-CGA against the compared methods based on ten matched runs.

Compared method	Settings	F1 (W/NS/L)	SHD (W/NS/L)	Runtime (W/NS/L)
PSX	24	19/5/0	19/5/0	24/0/0
EKGA-BN	24	19/5/0	19/5/0	24/0/0
AESL-GA	24	20/4/0	20/4/0	24/0/0
hybrid-SLA	24	19/5/0	18/6/0	24/0/0
MIGA	24	16/8/0	17/7/0	24/0/0
MAGABN	24	20/4/0	21/3/0	19/3/2
GTT	21	12/9/0	12/9/0	11/1/9
BS	21	16/5/0	15/6/0	11/3/7
IGES	21	14/7/0	15/5/1	18/0/3
saiyanH	24	22/2/0	23/1/0	13/1/10

methods GTT, BS, IGES, and saiyanH vary across network scales. The non-EA methods are significantly faster in several smaller settings, whereas the runtime advantage of CS-CGA becomes more apparent as the network scale increases.

5.2.4. Convergence analysis

Since the actual structure is assumed to be unknown during DAG learning, F1 score and SHD cannot be used for individual evaluation and selection. Existing scoring functions are insufficient to reflect the structural accuracy of a learned DAG relative to the actual structure, which is an inherent limitation of SS methods for structure learning from observational data. In the experiments, all methods use BIC to rank candidate DAG structures during search, as BIC reflects both the degree of data fitting and the structural complexity of a DAG. Fig. 3 shows the BIC convergence curves of CS-CGA and the six compared GA-based hybrid methods.

The convergence speed and stability are critical metrics for evaluating the global search efficiency and optimization capability of GAs. CS-CGA shows a clear optimization advantage over the compared GA-based hybrid methods in most cases. On seven networks, CS-CGA starts from higher initial BIC scores and maintains this advantage throughout the evolutionary process, indicating that its initialization strategy can generate candidate DAGs with a better trade-off between data fitting and model complexity. Although CS-CGA obtains lower initial BIC scores than MIGA on AS-500, AS-1000, and AS-3000, it rapidly approaches and eventually surpasses MIGA. The convergence results suggest that the constraint-search collaboration and MI guidance in CS-CGA help reduce the practical influence of the scoring limitation, where structurally different networks may receive close BIC scores and mislead the evolutionary search.

The above results validate the collaborative effectiveness of the PCR mechanism and the EEAM operator in CS-CGA. The PCR mechanism ensures an initial population with high fitness by limiting the search space to high-confidence regions in the early stages, which contributes to the higher starting points of convergence. Specifically, CS-CGA first constructs a high-confidence superstructure using a strict CI test threshold, so many unreliable candidate edges are excluded before the initial population is generated. The initial edges are then generated under MI guidance, which biases the initial DAGs toward stronger statistical dependencies. As a result, the initial individuals of CS-CGA commonly have higher BIC scores than those generated from broader or less selective search spaces. Even when the initial population quality is suboptimal, the EEAM operator enables the algorithm to escape local optima and rapidly search superior structures.

5.3. Analysis of the PCR mechanism

5.3.1. Analysis of PCR trigger behavior

An empirical analysis is conducted to examine the evolution of the search space during the GA process and the sensitivity of the PCR trigger to stochastic fluctuations. To this end, four of the eight benchmark BNs, namely Insurance, Alarm, Hailfinder, and Andes, are each evaluated on a dataset of 1000 samples over 10 runs with different random seeds.

As shown in Table 6, PCR is triggered an average of 18.1 to 27.0 times over 200 generations. The largest observed increase in α during one update is 0.00269. Hailfinder has the highest mean trigger count, but α increases from 0.00701 to 0.07430 through updates no larger than 0.00255. On the larger Andes network, 18.1 trigger events increase α from 0.00213 to 0.04744, while the relaxed superstructure expands from 624.3 to 1939.8 edges. These results indicate that PCR progressively expands the search space during the GA process, while the bounded update of α limits the effect of a trigger arising from stochastic fluctuations.

As shown in Fig. 4, the solid blue and dashed orange curves represent the mean value of α and the mean number of edges in the relaxed superstructure over 10 runs, respectively. The corresponding shaded regions indicate one standard deviation above and below each mean curve. Both curves show gradual stepwise growth. An increase in α relaxes the CI test threshold and can introduce additional candidate edges into the relaxed superstructure. The growth of curves occurs without abrupt changes, indicating that occasional stochastic fluctuations have a limited influence on the overall trend of search space expansion under PCR. As the GA approaches convergence, the increases generally become smaller and less frequent, which reduces the influence of PCR on convergence in the later generations. Even if a stochastic fluctuation triggers relaxation, the recovered edges only enter the relaxed superstructure as candidates. Candidate DAGs generated in the expanded search space remain subject to BIC-based fitness evaluation and selection, limiting potential adverse effects on the learned structure.

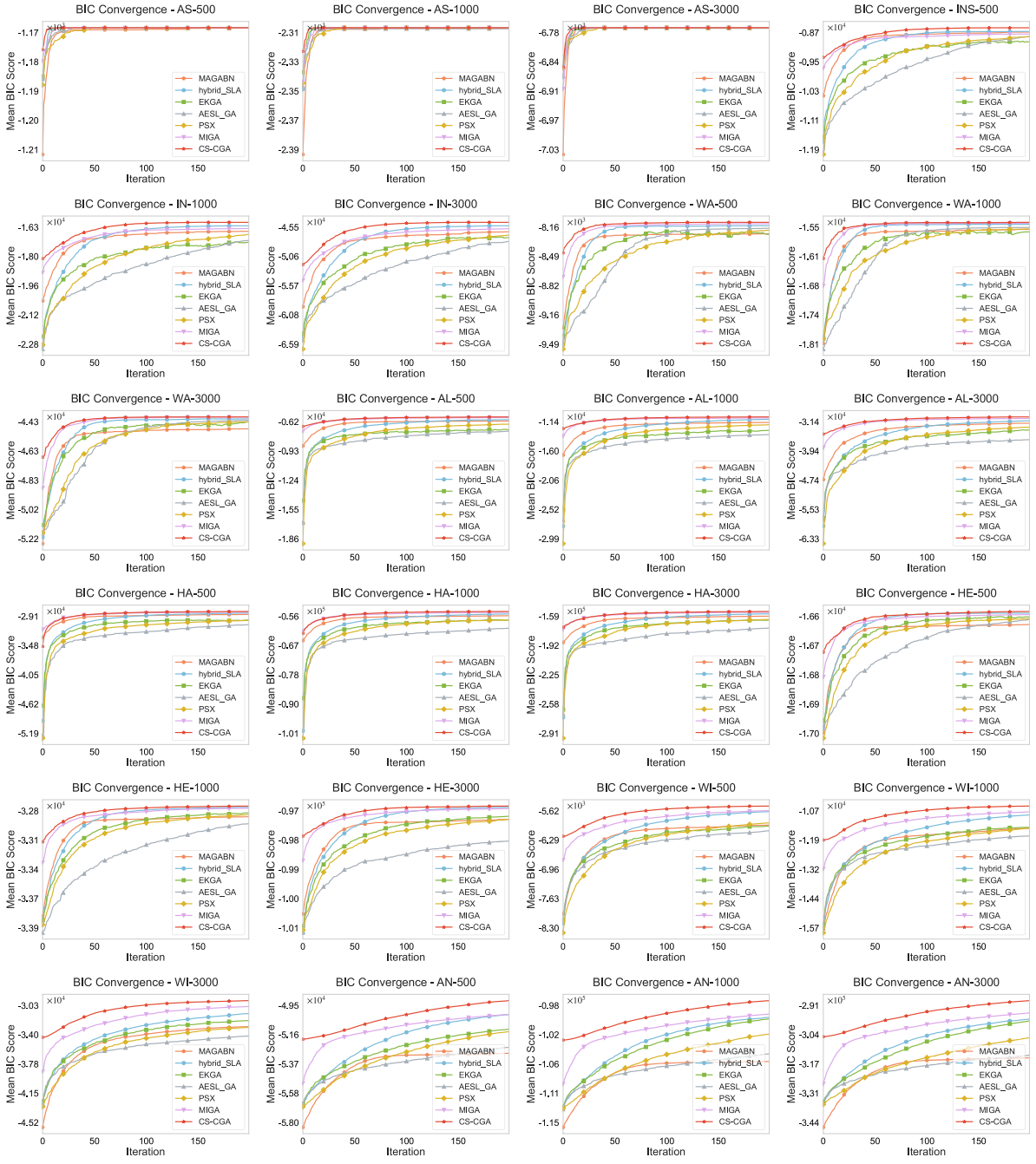
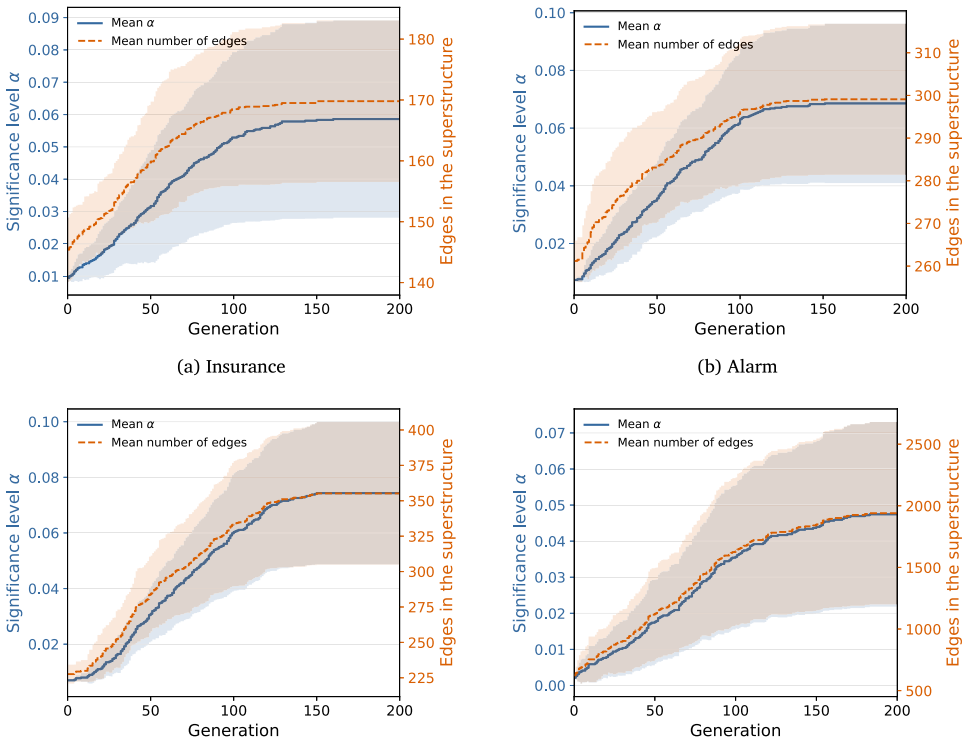


Fig. 3. Comparison of convergence characteristics between CS-CGA and other GA-based methods across 24 datasets derived from eight networks with sample sizes of 500, 1000, and 3000.

Table 6
PCR trigger statistics over 10 runs on four benchmark BNs with 1000 samples. Largest $\Delta\alpha$ denotes the largest observed single-update increase, while the remaining numerical values are averages over the 10 runs.

Network	Initial α	Triggers	Largest $\Delta\alpha$	Final α	Initial edges	Final edges
Insurance	0.00948	19.7	0.00257	0.05860	145.4	169.8
Alarm	0.00736	24.5	0.00269	0.06857	261.2	299.1
Hailfinder	0.00701	27.0	0.00255	0.07430	227.6	355.2
Andes	0.00213	18.1	0.00260	0.04744	624.3	1939.8



(a) Insurance (b) Alarm

(c) Hailfinder (d) Andes

Fig. 4. Evolution of the significance level and the relaxed superstructure under the PCR mechanism.

5.3.2. *Transferability assessment of the PCR mechanism*

To further assess the transferability of the proposed dynamic constraints, the PCR mechanism is embedded into two representative algorithms: hybrid-SLA and MIGA. The PCR-enhanced variants are compared with the original versions configured with various fixed significance levels ranging from $\alpha = 0.01$ to 1.00, as shown in Figs. 5 and 6.

For hybrid-SLA, the performance differences across fixed α values are minor for small networks. However, as network size increases, relaxed thresholds result in the retention of excessive false edges, leading to a sharp decline in the F1 score. In contrast, strict thresholds (e.g., $\alpha = 0.01$) perform better on medium-scale networks but still fall behind the PCR-enhanced version. On large-scale networks, the variant that uses dynamic constraint relaxation achieves the highest F1 scores, with its advantage becoming more pronounced as network complexity increases.

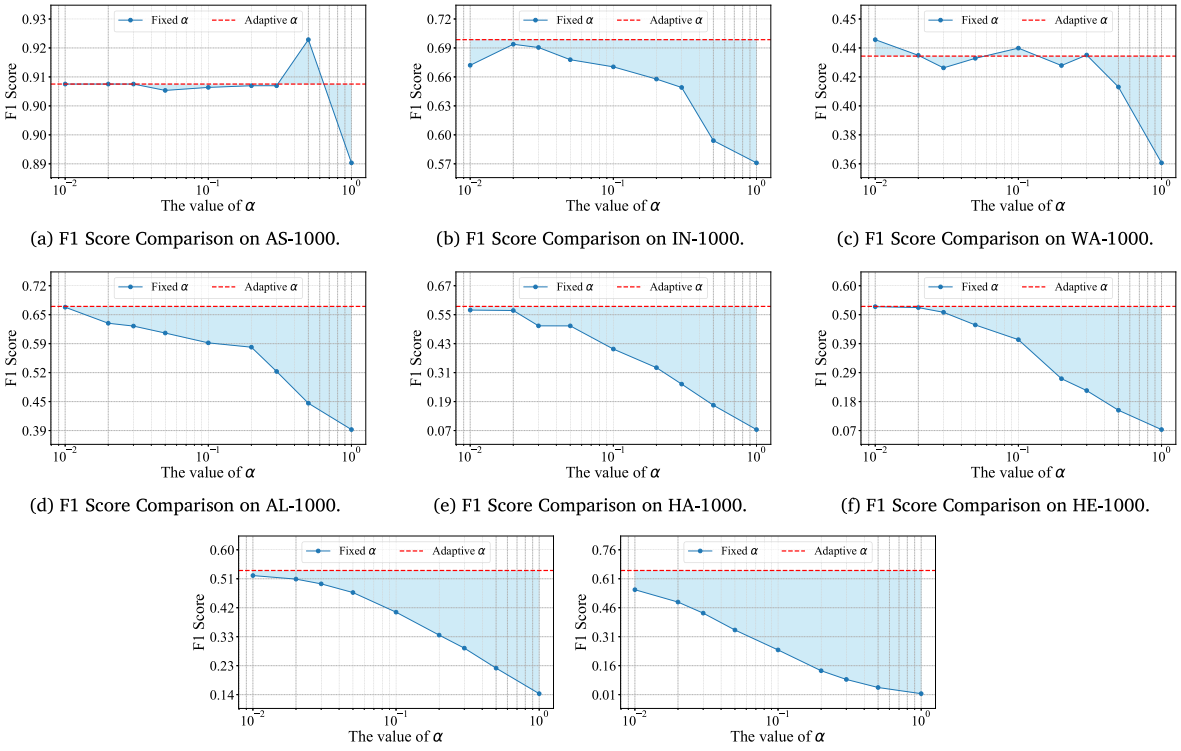
For MIGA, strict thresholds with $\alpha < 0.01$ achieve higher F1 scores than relaxed thresholds on half of the networks, including Insurance, Water, Hepar II, and Andes. However, strict thresholds are outperformed by more relaxed thresholds on the other three networks. In contrast, the PCR-embedded variant consistently yields the highest or near-optimal F1 scores across all cases. Even in the few instances where it slightly lags behind the best-performing fixed threshold configuration, its performance remains highly competitive and stable.

The results highlight the poor robustness of fixed thresholds in the CB phase, where performance varies markedly across networks of different scales and complexities. The proposed PCR mechanism addresses the limitation by dynamically recovering edges, ensuring stable performance across diverse algorithms and datasets.

5.4. *Ablation study on the contributions of PCR mechanism and EEAM operator to structural accuracy*

This section focuses on evaluating the contributions of the PCR mechanism and the EEAM operator to structural accuracy. To decouple the effects of progressive constraint, MI guidance, and adaptive mutation, we compare CS-CGA against five variant configurations. The results of the F1 score and SHD are presented in Fig. 7. The variants are named as follows:

- SR-Fix: Single-stage random mutation with a fixed significance level ($\alpha = 0.01$). It represents a standard GA without the proposed enhancements.
- SR-PCR: Single-stage random mutation with the PCR mechanism.
- SM-Fix: Single-stage MI-guided mutation with a fixed significance level ($\alpha = 0.01$).
- SM-PCR: Single-stage MI-guided mutation with the PCR mechanism.
- DR-PCR: Dual-stage random mutation of the EEAM operator with the PCR mechanism.

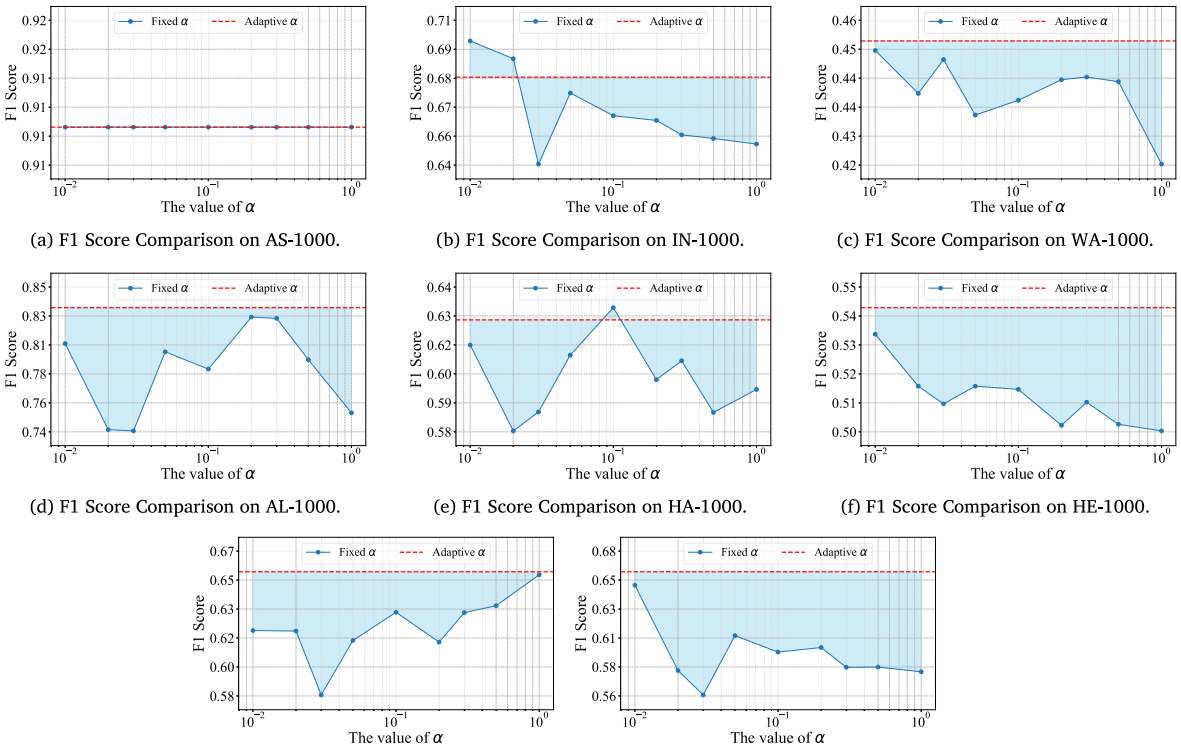


(a) F1 Score Comparison on AS-1000. (b) F1 Score Comparison on IN-1000. (c) F1 Score Comparison on WA-1000.

(d) F1 Score Comparison on AL-1000. (e) F1 Score Comparison on HA-1000. (f) F1 Score Comparison on HE-1000.

(g) F1 Score Comparison on WI-1000. (h) F1 Score Comparison on AN-1000.

Fig. 5. Comparison of PCR mechanism embedding and fixed significance levels in hybrid_SLA for F1 scores.

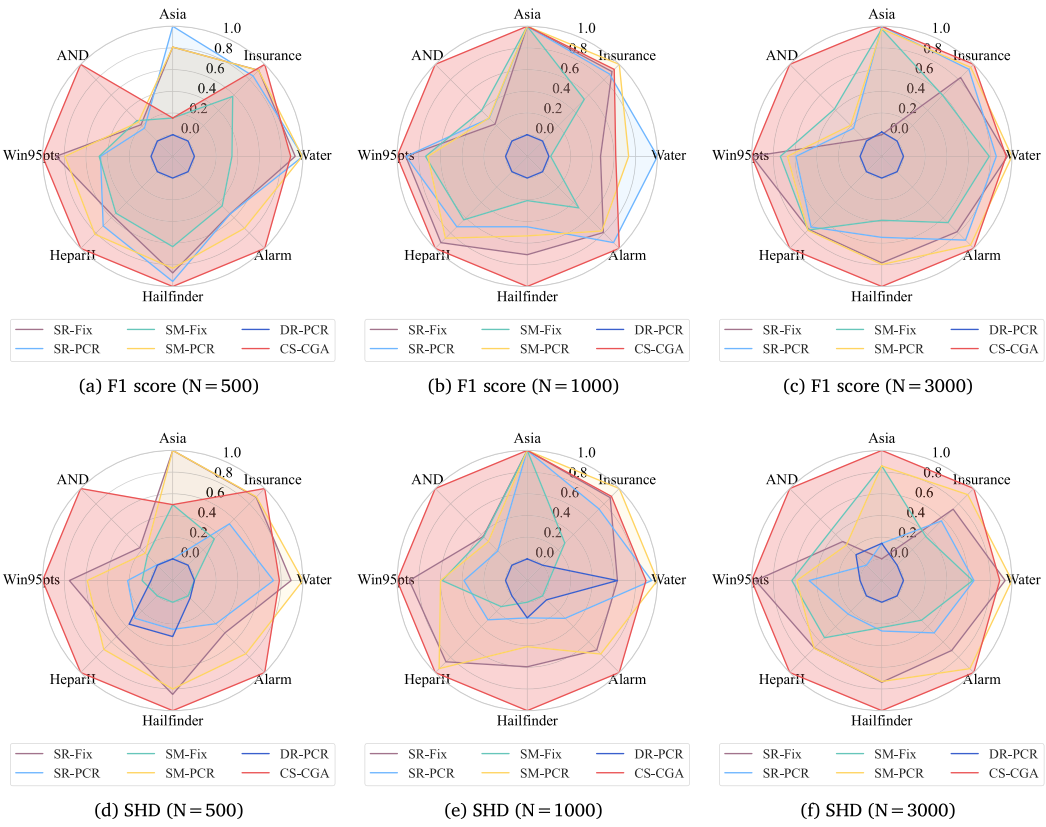


(a) F1 Score Comparison on AS-1000. (b) F1 Score Comparison on IN-1000. (c) F1 Score Comparison on WA-1000.

(d) F1 Score Comparison on AL-1000. (e) F1 Score Comparison on HA-1000. (f) F1 Score Comparison on HE-1000.

(g) F1 Score Comparison on WI-1000. (h) F1 Score Comparison on AN-1000.

Fig. 6. Comparison of PCR mechanism embedding and fixed significance levels in MIGA for F1 scores.



(a) F1 score (N = 500) (b) F1 score (N = 1000) (c) F1 score (N = 3000)

(d) SHD (N = 500) (e) SHD (N = 1000) (f) SHD (N = 3000)

Fig. 7. Ablation study results: radar charts comparing the structural accuracy (F1 score and SHD) of CS-CGA and its variants across different sample sizes (N = 500, 1000, and 3000).

- CS-CGA: The full model integrating the EEAM operator and the PCR mechanism. HSM is enabled by default as it yields identical accuracy.

As illustrated in Fig. 7, CS-CGA achieves the best performance across the majority of datasets in terms of both F1 scores and SHD. Variants utilizing the PCR mechanism, such as SR-PCR and SM-PCR, consistently outperform their fixed counterparts, SR-Fix and SM-Fix. The inferior performance of fixed-threshold variants confirms that a rigidly constrained search space imposes adverse effects on hybrid structure learning, resulting in the exclusion of candidate edges that may correspond to true causal dependencies under standard assumptions. Conversely, the improvements observed in variants with the PCR mechanism validate the capability of dynamic constraint relaxation to recover excluded candidate edges according to the evolutionary state.

The impact of MI guidance is evident from the comparisons between random and MI-guided variants. SM-PCR outperforms SR-PCR, indicating that MI guidance improves search accuracy. The MI-guided strategy avoids the blindness of random mutation and accelerates convergence toward promising regions of the search space.

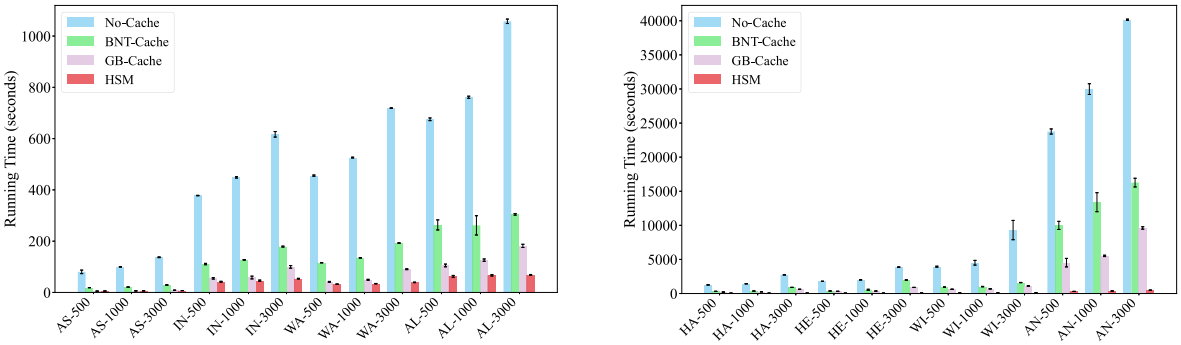
Finally, the comparison between CS-CGA and its counterpart (SM-PCR) with single-stage mutation emphasizes the necessity of the dual-stage mutation within the EEAM operator. While single-stage methods may become trapped in local optima, the second mutation in the expanded search space enhances exploration capability. The integration of progressive constraint, MI guidance, and adaptive mutation ensures a robust balance between local exploitation and global diversity, further contributing to the stability and accuracy of the algorithm.

5.5. Efficiency analysis of the HSM strategy

5.5.1. Ablation study of the HSM strategy

To assess the efficacy of the proposed acceleration strategy, an ablation study is conducted to compare the runtime efficiency across four scoring configurations. The specific strategies are defined as follows:

- No-Cache: A configuration without any score memoization, where the fitness function is fully recomputed for every individual in the population.
- BNT-Cache: A standard scoring cache strategy implementation that caches scores based on local parent-set structures(similar to the caching mechanism in the BNT toolbox).



(a) Runtime comparison on small-scale networks (smaller than Alarm).

(b) Runtime comparison on medium to large-scale networks (larger than Alarm).

Fig. 8. Runtime efficiency analysis of the HSM strategy compared to three caching configurations.

Table 7
Runtime comparison, cache size and hit rate for the global topology cache.

	Runtime Comparison (s)			Cache and Hit Rate Analysis		
Network	HSM	NoReuse	NoHash	Hit rate (%)	Hits / queries	Estimated size (KB)
Asia	**7.8**	14.6	15.5	79.9	32,104 / 40,200	309.8
Insurance	**41.3**	59.4	201.2	24.0	9668 / 40,200	5559.6
Water	**30.8**	47.3	189.7	23.7	9527 / 40,200	4440.1
Alarm	**52.7**	88.0	228.4	37.1	14,913 / 40,200	6130.2
Hailfinder	**91.7**	129.9	283.6	20.4	8187 / 40,200	9274.3
Hepar II	**81.0**	143.8	307.8	31.2	12,523 / 40,200	8276.7
Win95pts	**138.1**	219.1	348.9	25.5	10,258 / 40,200	15270.0
Andes	**382.2**	587.2	712.9	21.4	8607 / 40,200	54091.2

- GB-Cache: A hybrid strategy combining the global topology hash with the standard BNT-Cache.
- HSM (Proposed): The complete proposed strategy, which integrates the global topology hash with a dynamic local parent-set cache.

Runtime comparisons are visualized in the bar chart in Fig. 8. Since redundant score recomputation is the primary bottleneck in evolutionary structure learning, No-Cache exhibits substantially higher runtime across all datasets compared to the other methods. As expected, the runtime increases rapidly with network size and dataset scale. The introduction of BNT-Cache yields clear acceleration across most datasets, achieving an average speedup of approximately 4–6 times over No-Cache, thereby validating that the parent-set scoring cache can reduce redundant calculations.

By further incorporating global topology hashing, GB-Cache reduces runtime across all datasets compared to BNT-Cache, demonstrating that caching complete DAG scores enhances computational efficiency. Notably, the proposed HSM strategy achieves the best performance in all comparisons. Compared to BNT-Cache, HSM reduces runtime by more than 80% on most datasets. On the Andes-3000 dataset, HSM reduces runtime to just 2% of that required by BNT-Cache. The above results show that the hierarchical integration of global topology hashing and local parent-set caching reduces redundant score evaluations and improves scoring efficiency.

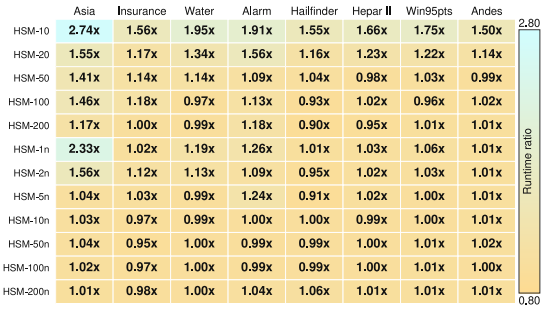
5.5.2. Analysis of global topology cache performance

To evaluate the efficiency and cache hit rate of global topology hashing, HSM, NoReuse, and NoHash are compared on eight benchmark BNs with 1000 samples. HSM uses the complete proposed strategy. NoReuse does not reuse cached complete DAG scores. NoHash retains the same complete DAG score cache but replaces hash-based retrieval with sequential matching. The left three columns of Table 7 compare the runtime of the three configurations, while the right three columns report the global topology cache statistics of HSM.

HSM achieves the lowest runtime on all eight networks. Without complete DAG score reuse, runtime increases by 41.7% to 86.8%. Replacing hash-based retrieval with sequential matching requires 1.87 to 6.16 times the runtime of HSM. The global cache hit rate ranges from 20.4% to 79.9%, corresponding to 8187 to 32,104 reused complete DAG scores. The estimated global cache size ranges from 309.8 KB on Asia to 54091.2 KB on Andes. These results show that complete DAG score reuse improves the scoring efficiency of CS-CGA and that hash-based storage and retrieval are more efficient for storing and retrieving DAG structures and their complete scores.

5.5.3. Analysis of the tradeoff between parent-set cache size and performance

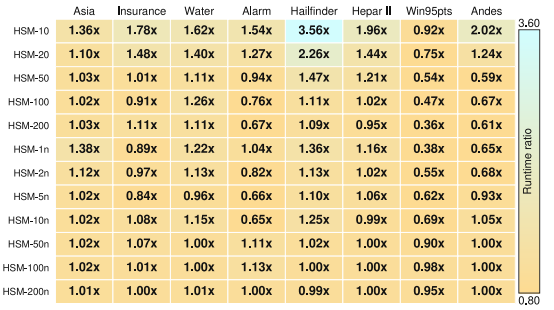
To further analyze the impact of local parent-set cache size on the efficiency of the HSM strategy, fixed cache sizes and cache sizes scaled by the number of nodes are compared on eight benchmark BNs with 1000 samples. HSM-large is a reference with an



	Asia	Insurance	Water	Alarm	Hailfinder	Hepar II	Win95pts	Andes
HSM-10	2.74x	1.56x	1.95x	1.91x	1.55x	1.66x	1.75x	1.50x
HSM-20	1.55x	1.17x	1.34x	1.56x	1.16x	1.23x	1.22x	1.14x
HSM-50	1.41x	1.14x	1.14x	1.09x	1.04x	0.98x	1.03x	0.99x
HSM-100	1.46x	1.18x	0.97x	1.13x	0.93x	1.02x	0.96x	1.02x
HSM-200	1.17x	1.00x	0.99x	1.18x	0.90x	0.95x	1.01x	1.01x
HSM-1n	2.33x	1.02x	1.19x	1.26x	1.01x	1.03x	1.06x	1.01x
HSM-2n	1.56x	1.12x	1.13x	1.09x	0.95x	1.02x	1.03x	1.01x
HSM-5n	1.04x	1.03x	0.99x	1.24x	0.91x	1.02x	1.00x	1.01x
HSM-10n	1.03x	0.97x	0.99x	1.00x	1.00x	0.99x	1.00x	1.01x
HSM-50n	1.04x	0.95x	1.00x	0.99x	0.99x	1.00x	1.01x	1.02x
HSM-100n	1.02x	0.97x	1.00x	0.99x	0.99x	1.00x	1.01x	1.00x
HSM-200n	1.01x	0.98x	1.00x	1.04x	1.06x	1.01x	1.01x	1.01x

Runtime ratio: 2.80 / 0.80

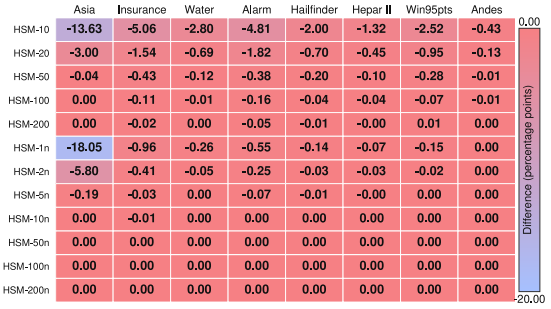
(a) CS-CGA runtime ratio.



	Asia	Insurance	Water	Alarm	Hailfinder	Hepar II	Win95pts	Andes
HSM-10	1.36x	1.78x	1.62x	1.54x	3.56x	1.96x	0.92x	2.02x
HSM-20	1.10x	1.48x	1.40x	1.27x	2.26x	1.44x	0.75x	1.24x
HSM-50	1.03x	1.01x	1.11x	0.94x	1.47x	1.21x	0.54x	0.59x
HSM-100	1.02x	0.91x	1.26x	0.76x	1.11x	1.02x	0.47x	0.67x
HSM-200	1.03x	1.11x	1.11x	0.67x	1.09x	0.95x	0.36x	0.61x
HSM-1n	1.38x	0.89x	1.22x	1.04x	1.36x	1.16x	0.38x	0.65x
HSM-2n	1.12x	0.97x	1.13x	0.82x	1.13x	1.02x	0.55x	0.68x
HSM-5n	1.02x	0.84x	0.96x	0.66x	1.10x	1.06x	0.62x	0.93x
HSM-10n	1.02x	1.08x	1.15x	0.65x	1.25x	0.99x	0.69x	1.05x
HSM-50n	1.02x	1.07x	1.00x	1.11x	1.02x	1.00x	0.90x	1.00x
HSM-100n	1.02x	1.01x	1.00x	1.13x	1.00x	1.00x	0.98x	1.00x
HSM-200n	1.01x	1.00x	1.01x	1.00x	0.99x	1.00x	0.95x	1.00x

Runtime ratio: 3.60 / 0.80

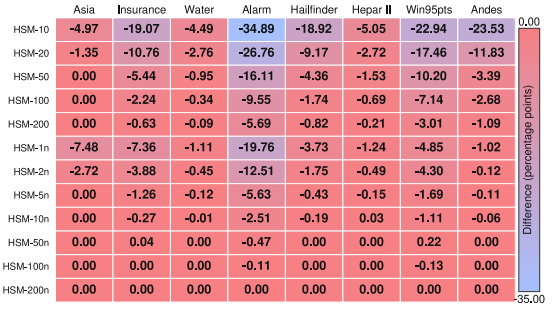
(b) AESL-GA runtime ratio.



	Asia	Insurance	Water	Alarm	Hailfinder	Hepar II	Win95pts	Andes
HSM-10	-13.63	-5.06	-2.80	-4.81	-2.00	-1.32	-2.52	-0.43
HSM-20	-3.00	-1.54	-0.69	-1.82	-0.70	-0.45	-0.95	-0.13
HSM-50	-0.04	-0.43	-0.12	-0.38	-0.20	-0.10	-0.28	-0.01
HSM-100	0.00	-0.11	-0.01	-0.16	-0.04	-0.04	-0.07	-0.01
HSM-200	0.00	-0.02	0.00	-0.05	-0.01	-0.00	0.01	0.00
HSM-1n	-18.05	-0.96	-0.26	-0.55	-0.14	-0.07	-0.15	0.00
HSM-2n	-5.80	-0.41	-0.05	-0.25	-0.03	-0.03	-0.02	0.00
HSM-5n	-0.19	-0.03	0.00	-0.07	-0.01	-0.00	0.00	0.00
HSM-10n	0.00	-0.01	0.00	0.00	0.00	0.00	0.00	0.00
HSM-50n	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
HSM-100n	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
HSM-200n	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

Difference (percentage points): 0.00 / -20.00

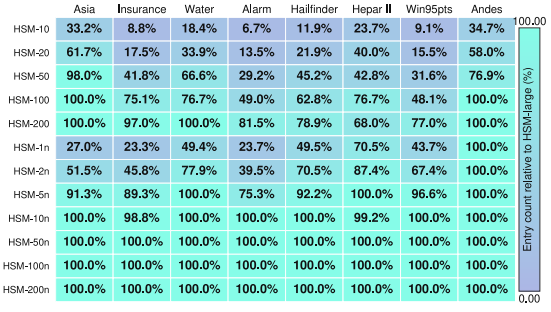
(c) CS-CGA difference in local cache hit rate.



	Asia	Insurance	Water	Alarm	Hailfinder	Hepar II	Win95pts	Andes
HSM-10	-4.97	-19.07	-4.49	-34.89	-18.92	-5.05	-22.94	-23.53
HSM-20	-1.35	-10.76	-2.76	-26.76	-9.17	-2.72	-17.46	-11.83
HSM-50	0.00	-5.44	-0.95	-16.11	-4.36	-1.53	-10.20	-3.39
HSM-100	0.00	-2.24	-0.34	-9.55	-1.74	-0.69	-7.14	-2.68
HSM-200	0.00	-0.63	-0.09	-5.69	-0.82	-0.21	-3.01	-1.09
HSM-1n	-7.48	-7.36	-1.11	-19.76	-3.73	-1.24	-4.85	-1.02
HSM-2n	-2.72	-3.88	-0.45	-12.51	-1.75	-0.49	-4.30	-0.12
HSM-5n	0.00	-1.26	-0.12	-5.63	-0.43	-0.15	-1.69	-0.11
HSM-10n	0.00	-0.27	-0.01	-2.51	-0.19	0.03	-1.11	-0.06
HSM-50n	0.00	0.04	0.00	-0.47	0.00	0.00	0.22	0.00
HSM-100n	0.00	0.00	0.00	-0.11	0.00	0.00	-0.13	0.00
HSM-200n	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

Difference (percentage points): 0.00 / -35.00

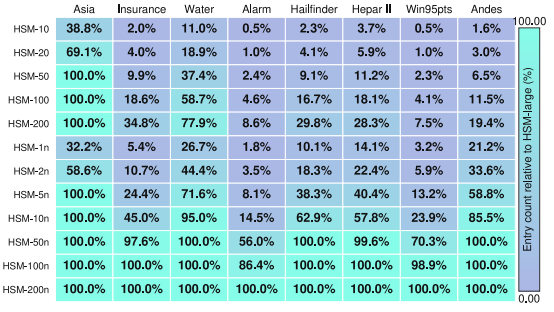
(d) AESL-GA difference in local cache hit rate.



	Asia	Insurance	Water	Alarm	Hailfinder	Hepar II	Win95pts	Andes
HSM-10	33.2%	8.8%	18.4%	6.7%	11.9%	23.7%	9.1%	34.7%
HSM-20	61.7%	17.5%	33.9%	13.5%	21.9%	40.0%	15.5%	58.0%
HSM-50	98.0%	41.8%	66.6%	29.2%	45.2%	42.8%	31.6%	76.9%
HSM-100	100.0%	75.1%	76.7%	49.0%	62.8%	76.7%	48.1%	100.0%
HSM-200	100.0%	97.0%	100.0%	81.5%	78.9%	68.0%	77.0%	100.0%
HSM-1n	27.0%	23.3%	49.4%	23.7%	49.5%	70.5%	43.7%	100.0%
HSM-2n	51.5%	45.8%	77.9%	39.5%	70.5%	87.4%	67.4%	100.0%
HSM-5n	91.3%	89.3%	100.0%	75.3%	92.2%	100.0%	96.6%	100.0%
HSM-10n	100.0%	98.8%	100.0%	100.0%	100.0%	99.2%	100.0%	100.0%
HSM-50n	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%
HSM-100n	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%
HSM-200n	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%

Entry count relative to HSM-large (%): 100.00 / 0.00

(e) CS-CGA relative local parent-set entry count.



	Asia	Insurance	Water	Alarm	Hailfinder	Hepar II	Win95pts	Andes
HSM-10	38.8%	2.0%	11.0%	0.5%	2.3%	3.7%	0.5%	1.6%
HSM-20	69.1%	4.0%	18.9%	1.0%	4.1%	5.9%	1.0%	3.0%
HSM-50	100.0%	9.9%	37.4%	2.4%	9.1%	11.2%	2.3%	6.5%
HSM-100	100.0%	18.6%	58.7%	4.6%	16.7%	18.1%	4.1%	11.5%
HSM-200	100.0%	34.8%	77.9%	8.6%	29.8%	28.3%	7.5%	19.4%
HSM-1n	32.2%	5.4%	26.7%	1.8%	10.1%	14.1%	3.2%	21.2%
HSM-2n	58.6%	10.7%	44.4%	3.5%	18.3%	22.4%	5.9%	33.6%
HSM-5n	100.0%	24.4%	71.6%	8.1%	38.3%	40.4%	13.2%	58.8%
HSM-10n	100.0%	45.0%	95.0%	14.5%	62.9%	57.8%	23.9%	85.5%
HSM-50n	100.0%	97.6%	100.0%	56.0%	100.0%	99.6%	70.3%	100.0%
HSM-100n	100.0%	100.0%	100.0%	86.4%	100.0%	100.0%	98.9%	100.0%
HSM-200n	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%

Entry count relative to HSM-large (%): 100.00 / 0.00

(f) AESL-GA relative local parent-set entry count.

Fig. 9. Effects of cache size on runtime, local cache hit rate, and local parent-set entries for CS-CGA and AESL-GA.

unlimited cache size and no cache replacement. Fig. 9(a), (c), and (e) show the runtime ratio, the difference in local cache hit rate in percentage points, and the relative local parent-set entry count for CS-CGA compared with HSM-large. Fig. 9(b), (d), and (f) show the corresponding results for AESL-GA.

For CS-CGA, small cache sizes cause frequent replacement of local parent-set scores, reducing hit rates and increasing repeated score evaluations. HSM-10 is 1.50 to 2.74 times slower than HSM-large, whereas HSM-50, HSM-100, and HSM-200 achieve hit rates close to those of HSM-large on most networks with substantially fewer entries. On the Hepar II network, HSM-50 retains only 42.8% of the reference entries while maintaining a hit rate close to that of HSM-large. A similar pattern is observed on Water, Alarm, and Win95pts.

The size of local cache requirements depends on more than the number of nodes. Water has multivalued variables and complex CPTs, making repeated local score evaluations costly. In contrast, HSM-1n allocates 223 entries per node on the Andes network and achieves a hit rate close to that of HSM-large.

The AESL-GA results further indicate that cache size requirements are also related to the diversity of parent sets encountered during the search. Compared with CS-CGA, AESL-GA evaluates more distinct parent sets and requires larger local cache sizes on Alarm, Win95pts, and Andes. On Alarm, HSM-10 retains only 0.5% of the reference entries and achieves a hit rate 34.89 percentage points lower than that of HSM-large. On Andes, HSM-10 and HSM-200 retain only 1.6% and 19.4% of the reference entries, respectively.

Nevertheless, increasing the cache size cannot necessarily improve efficiency because entries with low reuse increase local cache lookup overhead. On the Andes network, the runtime of AESL-GA decreases when the cache size increases from HSM-10 to HSM-50. As the cache size increases further, the gains in hit rate become marginal, while the runtime gradually increases to a level close to

that of HSM-large. In contrast, HSM-200 retains only 19.4% of the local parent-set cache entries of HSM-large and reduces runtime by 39.0%.

Overall, the results show that a small HSM cache size leads to frequent replacements and repeated score evaluations, whereas an unbounded or excessively large cache retains many entries with low reuse. By limiting the local cache size of each node and applying cyclic replacement, HSM maintains high local cache hit rates with substantially fewer actual parent-set entries than HSM-large, thereby improving the efficiency of structure search.

6. Conclusion

This paper proposes a collaborative optimization method to address the limitations of static search spaces and inefficient fitness evaluations in hybrid BNSL. The proposed method integrates a progressive constraint relaxation mechanism and an exploration-enhanced adaptive mutation operator to recover lost potential causal relationships and dynamically balance global exploration. Additionally, the hierarchical score memoization strategy is incorporated to reduce computational overhead by eliminating redundant calculations. Experimental results demonstrate statistically significant advantages of the proposed method in structural accuracy and runtime, while also showing favorable convergence behavior compared with the GA-based methods. Future work will further investigate more robust trigger conditions for the PCR mechanism, such as signals indicating that the search has stagnated or become trapped in a local optimum. The collaborative process will also be extended to handle continuous variables and explore broader applicability to real-world causal discovery tasks.

CRediT authorship contribution statement

Zuowei He: Writing – original draft, Methodology; **Hengyang Lu:** Resources, Investigation; **Shuwei Zhu:** Data curation; **Yunyang Shi:** Investigation; **Wei Fang:** Supervision, Funding acquisition; **Kalyanmoy Deb:** Writing – review & editing, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant 62473176, in part by the Postgraduate Research & Practice Innovation Program of Jiangsu Province (KYCX24_2642), in part by the Foundation for Interdisciplinary Project of Jiangnan University (No. 2026JC06B, No. 2026JC01A), and in part by the fund of Laboratory for Advanced Computing and Intelligence Engineering.

Code availability

The CS-CGA code and data are publicly available at https://github.com/zuowhe/CS-CGA.

Data availability

No data was used for the research described in the article.

References

[1] W. Xiao, H. Wang, M. He, N. Wang, Boosting causal structure learning: an asymmetric exponential modulation gaussian-based adaptive sample reweighting framework, in: Proceedings of the AAAI Conference on Artificial Intelligence, 39, 2025, pp. 21661–21669.
[2] S. Rey, S.S. Saboksayr, G. Mateos, Non-negative weighted DAG structure learning, in: ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2025, pp. 1–5.
[3] B. Lehmann, S. White, A Bayesian multilevel model for populations of networks using exponential-family random graphs, Statist. Comput. 34 (4) (2024) 136.
[4] C. Liu, D.R. Kowal, M. Vannucci, Dynamic and robust Bayesian graphical models, Statist. Comput. 32 (6) (2022) 105.
[5] Y. Xu, K. Li, Y. Liu, Quantitative analysis of risk propagation in urban rail transit: a novel ensemble learning method based on the structure of Bayesian Network, Reliab. Eng. Syst. Saf. 263 (2025) 111280.
[6] Y. Zhao, J. Jia, DAGSLAM: causal Bayesian network structure learning of mixed type data and its application in identifying disease risk factors, BMC Med. Res. Methodol. 25 (1) (2025) 154.
[7] N.K. Kitson, A.C. Constantinou, Z. Guo, Y. Liu, K. Chobtham, A survey of Bayesian network structure learning, Artif. Intell. Rev. 56 (8) (2023) 8721–8814.
[8] D.M. Chickering, D. Heckerman, C. Meek, Large-Sample learning of Bayesian networks is NP-hard, J. Mach. Learn. Res. 5 (2004) 1287–1330.
[9] P. Spirtes, C. Glymour, An algorithm for fast recovery of sparse causal graphs, Soc. Sci. Comput. Rev. 9 (1) (1991) 62–72.
[10] I. Tsamardinos, C.F. Aliferis, A. Statnikov, Time and sample efficient discovery of Markov blankets and direct causal relations, in: Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '03, Association for Computing Machinery, New York, NY, USA, 2003, p. 673–678.
[11] R. Yehezkel, B. Lerner, Bayesian network structure learning by recursive autonomy identification, J. Mach. Learn. Res. 10 (2009) 1527–1570.
[12] S. Gheisari, M.R. Meybodi, BNC-PSO: structure learning of Bayesian networks by particle swarm optimization, Inform. Sci. 348 (2016) 272–289.
[13] L. Chen, T. Ban, X. Wang, D. Lyu, H. Chen, Mitigating prior errors in causal structure learning: a resilient approach via Bayesian networks, IEEE Trans. Pattern Anal. Mach. Intell. 47 (12) (2025) 10990–11002.

[14] X. Xie, B. Xie, D. Xiong, M. Hou, J. Zuo, G. Wei, J. Chevallier, New theoretical ISM-K2 Bayesian network model for evaluating vaccination effectiveness, J. Ambient Intell. Humanized Comput. 14 (9) (2023) 12789–12805.
[15] X. Gao, X. Yan, Z. Wang, X. Liu, Bayesian network structure learning algorithm based on score increment and reduction, in: 2023 8th International Conference on Control and Robotics Engineering (ICCRE), 2023, pp. 1–5.
[16] Z. Wang, X. Gao, X. Tan, X. Liu, Learning Bayesian networks using A* search with ancestral constraints, Neurocomputing 451 (2021) 107–124.
[17] C. Wang, X. Gao, X. Li, B. Li, K. Wan, Finding community structure in Bayesian networks by heuristic K-standard deviation method, Futur. Gener. Comput. Syst. 158 (2024) 556–568.
[18] W. Gao, M. Zhi, Y. Ke, X. Wang, Y. Zhuo, A. Liu, Y. Yang, Bayesian network structure learning based on HC-PSO algorithm, J. Intell. Fuzzy Syst. 46 (2) (2024) 4347–4359.
[19] J.P.A.F. Campos, I.G. Machado, M. Bessani, Multi-Agent Genetic Algorithm for Bayesian networks structural learning, Knowl.-Based Syst. 310 (2025) 113025.
[20] M. Gasse, A. Aussem, H. Elghazel, A hybrid algorithm for Bayesian network structure learning with application to multi-label learning, Expert Syst. Appl. 41 (15) (2014) 6755–6772.
[21] S. Lee, S.B. Kim, Parallel simulated annealing with a greedy algorithm for Bayesian network structure learning, IEEE Trans. Knowl. Data Eng. 32 (6) (2020) 1157–1166.
[22] W. Zhang, W. Fang, J. Sun, Q. Chen, Learning Bayesian networks structures with an effective knowledge-driven GA, in: 2020 IEEE Congress on Evolutionary Computation (CEC), 2020, pp. 1–8.
[23] I. Tsamardinos, L.E. Brown, C.F. Aliferis, The max-min hill-climbing Bayesian network structure learning algorithm, Mach. Learn. 65 (1) (2006) 31–78.
[24] A.C. Constantinou, Learning Bayesian networks that enable full propagation of evidence, IEEE Access 8 (2020) 124845–124856.
[25] S. Jose, S. Liu, S. Louis, S. Dascalu, Towards a hybrid approach for evolving Bayesian networks using genetic algorithms, in: 2019 IEEE 31st International Conference on Tools With Artificial Intelligence (ICTAI), 2019, pp. 705–712.
[26] K. Yan, W. Fang, H. Lu, X. Zhang, J. Sun, X. Wu, Mutual Information-Guided GA for Bayesian network structure learning, IEEE Trans. Knowl. Data Eng. 35 (2023) 8282–8299.
[27] J. Dai, J. Ren, W. Du, Decomposition-based Bayesian network structure learning algorithm using local topology information, Knowl.-Based Syst. 195 (2020) 105602.
[28] C. Contaldi, F. Vafaee, P.C. Nelson, The role of crossover operator in Bayesian network structure learning performance: a comprehensive comparative study and new insights, in: Proceedings of the Genetic and Evolutionary Computation Conference, 2017, pp. 769–776.
[29] R.W. Robinson, Counting labeled acyclic digraphs, New Dir. Theory Graphs (1973) 239–273.
[30] J. Cheng, R. Greiner, J. Kelly, D. Bell, W. Liu, Learning Bayesian networks from data: an information-theory based approach, Artif. Intellig. 137 (1–2) (2002) 43–90.
[31] F. Pernkopf, J.A. Bilmes, Efficient heuristics for discriminative structure learning of Bayesian network classifiers, J. Mach. Learn. Res. 11 (2010) 2323–2360.
[32] L.M. de Campos, A scoring function for learning Bayesian networks based on mutual information and conditional independence tests, J. Mach. Learn. Res. 7 (2006) 2149–2187.
[33] G. Li, X. Gao, R. Di, DBN structure learning based on MI-BPSO algorithm, in: 2014 IEEE/ACIS 13th International Conference on Computer and Information Science (ICIS), 2014, pp. 245–250.
[34] J. Dai, J. Ren, W. Du, V. Shikhin, J. Ma, An improved evolutionary approach-based hybrid algorithm for Bayesian network structure learning in dynamic constrained search space, Neural Comput. Appl. 32 (5) (2020) 1413–1434.
[35] G.F. Cooper, E. Herskovits, A Bayesian method for the induction of probabilistic networks from data, Mach. Learn. 9 (4) (1992) 309–347.
[36] D. Heckerman, D. Geiger, D.M. Chickering, Learning Bayesian networks: the combination of knowledge and statistical data, Mach. Learn. 20 (3) (1995) 197–243.
[37] G. Schwarz, Estimating the dimension of a model, Annals Stat. 6 (1978) 461–464.
[38] J. Suzuki, A construction of Bayesian networks from databases based on an MDL principle, in: D. Heckerman, A. Mamdani (Eds.), Uncertainty in Artificial Intelligence, Morgan Kaufmann, 1993, pp. 266–273.
[39] Z. Liu, B. Malone, C. Yuan, Empirical evaluation of scoring functions for Bayesian network model selection, BMC Bioinformatics 13 (Suppl 15) (2012) S14.
[40] J. Cheng, D.A. Bell, W. Liu, An algorithm for Bayesian network construction from data, in: Proceedings of the Sixth International Workshop on Artificial Intelligence and Statistics, R1 of *Proceedings of Machine Learning Research*, PMLR, 1997, pp. 83–90.
[41] D.M. Chickering, Learning equivalence classes of Bayesian-network structures, J. Mach. Learn. Res. 2 (2002) 445–498.
[42] X. Zhao, W. Wan, Z. Fang, IGES-RCI: improved greedy equivalence search and recursive causal inference for industrial equipment failure prediction, IEEE Trans. Knowl. Data Eng. 37 (10) (2025) 5983–5993.
[43] E. Perrier, S. Imoto, S. Miyano, Finding optimal Bayesian network given a super-structure, J. Mach. Learn. Res. 9 (74) (2008) 2251–2286.
[44] P. Nandy, A. Hauser, M.H. Maathuis, High-dimensional consistency in score-based and hybrid structure learning, Annals Stat. 46 (6A) (2018) 3151–3183.
[45] A.C. Constantinou, Learning Bayesian networks with the Saiyan algorithm, ACM Trans. Knowl. Disc. Data 14 (4) (2020) 44:1–44:21.
[46] P.C. Pinto, A. Nagele, M. Dejori, T.A. Runkler, J.M.C. Sousa, Using a Local Discovery Ant Algorithm for Bayesian network structure learning, IEEE Trans. Evol. Comput. 13 (4) (2009) 767–779.
[47] B. Sun, Y. Zhou, J. Wang, W. Zhang, A new PC-PSO algorithm for Bayesian network structure learning with structure priors, Expert Syst. Appl. 184 (2021) 115237.
[48] A. Tonda, E. Lutton, G. Squillero, P.-H. Wuillemin, A memetic approach to Bayesian network structure learning, in: Applications of Evolutionary Computation, 7835 of *Lecture Notes in Computer Science*, Springer, Berlin, Heidelberg, 2013, pp. 102–111.
[49] C. Contaldi, F. Vafaee, P.C. Nelson, Bayesian network hybrid learning using an elite-guided genetic algorithm, Artif. Intell. Rev. 52 (2019) 245–272.
[50] K.P. Murphy, The Bayes Net Toolbox for MATLAB, in: Computing Science and Statistics, 2001.